CHURCH OF MALWARE / FIELD MANUAL



Intermediate RE with SPECTRA

Follow the bytes. Test an idea. Keep the evidence.

Build repeatable investigations with types, harnesses, process models, history, trace evidence, Android and five deeper CTF campaigns.

EDITION 3 / 11 NOVEMBER 2026
SPECTRA 0.18 / BENIGN PRACTICE FILES INCLUDED
Training Ground edition / campaigns 001-008

Contents

1. Turn a promising lead into a repeatable investigation 4

2. Loader reasoning and ARM static PIE 6

3. Indirect calls, switches, and honest coverage 8

4. Recognize library work without erasing application logic 10

5. Recover structures and propagate useful corrections 12

6. Call a native routine with a controlled contract 14

7. Batches, watchpoints, checkpoints, and negative controls 16

8. Verified history: reason backward within a bounded routine 18

9. Trace provenance, persistent segments and event alignment 20

10. Windows recipes: supply state without inventing reality 23

11. Linux experiments: descriptors, bytes, and missing services 25

12. ARM MMIO and portable hardware reasoning 27

13. Recovery as a chain of evidence 29

14. Managed clients: methods, labels, and a useful harness plan 31

15. Android: connect package, DEX, source and native evidence 33

16. Firmware and patch review as connected investigations 36

17. The intermediate casebook: contracts, counterexamples and handoffs 38

18. Ghost Circuit: one image, connected reasoning 40

19. Hollow Signal: a decoder, a parser and a boundary proof 48

20. Iron Veil: collisions, indirect calls and missing coverage 59

21. Black Loom: reconstruction, rebasing and incomplete output 70

22. Puppet Core: recovering a virtual instruction language 82

23. From reconstructed images to defensible interpreter models 95

24. Build a research package someone else can use 97

25. Sources and laboratory notes 99

1. Turn a promising lead into a repeatable investigation

This manual assumes you can identify a file, distinguish a VA from a file offset, follow a reference, read a simple loop, and save an investigation. We will now work on the difficult middle: imperfect metadata, ambiguous control flow, missing state, competing types, and results that change when your assumptions change.

Keep the beginner manual nearby for control definitions. This book uses SPECTRA 0.18 and the same benign practice pack. The objective is a defensible research workflow that stays in the browser wherever the current engines support it. Full operating systems, native host-process attachment, managed execution, and arbitrary device emulation are not implied by an engine dropdown.

Define the experiment before the setup

Replace “run the binary” with a contract: “At entry E, with ABI A, memory M, arguments I, and modeled services S, does this routine produce O before stop condition T?” This describes what can be reproduced. It also exposes the missing inputs before you spend an hour clicking Continue.

```
Artifact: exact SHA-256 and provenance
Entry: VA, function boundary evidence, load bias
Inputs: arguments, buffers, registers, globals
Environment: model, dependencies, hooks, virtual files
Prediction: output bytes, return, or branch outcome
Stop: return / exit / breakpoint / budget / fault
Observation: actual output and retained evidence
Residual uncertainty: what this setup cannot establish
```

Separate four levels of confidence

Stored fact: a byte range and hash. **Recovered relationship:** an instruction points at a string or target under the selected mapping. **Analyst interpretation:** this appears to be a record parser. **Observed model behavior:** the parser returned a particular value for supplied bytes. Each layer builds on earlier evidence but can introduce assumptions.

Use names to organize, not to erase uncertainty. `parse_record_candidate` is useful when you have two loads and a comparison. Rename to `validate_record_header` after callers and failure paths support that role. Preserve rejected hypotheses if they explain a consequential decision.

Choose a stopping rule

An intermediate analyst should be able to stop without “understanding everything.” For a decoder, the finish line may be a reproduced output with source range and parameters. For a patch review, it may be a confirmed changed condition plus its covered callers. For a client method, it may be an invocation recipe with explicit dependencies and untested server semantics.

Exercise: Rewrite “this binary encrypts data” as a testable question. One answer is: “Does function F transform the 32-byte supplied buffer into the predicted output when given key K and IV V under this ABI?” That tests a specific transformation. Identifying an algorithm or security property needs additional evidence.

2. Loader reasoning and ARM static PIE

A wrong mapping can make correct bytes tell the wrong story. Verify where file bytes become memory, which regions are zero-filled, where execution begins, and who applies relocations. For ELF, program headers describe loading; `p_filesz` and `p_memsz` can differ, with the extra loadable memory initialized to zero. The ELF program-loading specification defines these relationships. SPECTRA supports a selected subset rather than every ELF configuration.

Case file: watch a pointer relocate

1. Save your case. In Debugger lab choose ARM32 and **Load ARM PIE demo**.
2. Choose **Use detected setup** and inspect the load check, relocated entry, and relocation count.
3. Under **Load base, raw bytes & session assumptions**, verify the default ARM PIE load bias 0x10000000 and **Program startup** relocation ownership.
4. Initialize. Read four bytes at VA 0x10001000. Expect the little-endian word 0x1020 before startup relocation.
5. Add a four-byte write watchpoint there, run, and inspect the write. Expect 0x10001020 after startup adds the bias.
6. Remove the watchpoint and continue. The benign fixture writes “pie” to captured stdout and exits zero.

The arithmetic matters: 0x1020 + 0x10000000 = 0x10001020. The location of the pointer is 0x10001000; the value stored there points somewhere else. Record both. A screenshot showing one address without saying which role it has loses the key fact.

Relocation has an owner

In Program startup mode, SPECTRA maps the admitted static PIE but lets its startup routine adjust pointers. The alternative **Debugger applies REL · alternate entry** applies supported relative relocations before execution and requires a different start address. It is for a justified alternate-entry experiment, not a shortcut that performs all initialization.

If you apply relocations first and then re-enter a self-relocator, the bias may be added twice. Conversely, starting after initialization without supplying its effects can leave stale pointers. Constructors, global state, TLS, and argument context are separate from relocation. Passing one loader check does not supply them all.

Change the bias carefully

As a controlled follow-up, choose another aligned, non-overlapping load bias accepted by preflight, keep Program startup ownership, and initialize again. Recompute the watched location as bias + 0x1000 and predict the target as bias + 0x1020. Save both recipes and outputs. Do not reuse a breakpoint at the old loaded address.

The current ARM/Thumb path admits little-endian EABI5 static PIE with bounded local R_ARM_RELATIVE records and no required external interpreter/shared libraries. Unsupported relocation kinds remain blockers. This is not general dynamic linking or AArch64/x64 PIE support.

File-to-VA verification

For a file-backed load segment, a local mapping is `VA = segment VA + (file offset - segment offset) + load bias`. Apply it only within that segment's file-backed interval. The reverse mapping can fail for BSS or newly allocated memory. State the segment and bias with every derived address.

Deliverable: a before/after memory capture, relocation ownership, bias, pointer-storage address, pointer value, stop reason, and a sentence explaining why two distinct addresses are involved.

Use the ledger as a falsifiable conversion

File structure → Address mapping ledger performs exact unsigned 64-bit conversions for declared PE, ELF and thin Mach-O regions, the current raw range and captured debugger mappings. Extended ELF program-header numbering (PN_XNUM) is supported in the mapping parser. That parser capability is separate from loader admission in a particular debugger architecture.

Choose the mapping source and coordinate type explicitly. Resolve a known byte, inspect its original bytes when available and reverse the conversion. Preserve overlapping candidates instead of selecting a convenient unique answer. Test both the last file-backed byte and the first zero-filled byte in a segment: that boundary catches formulas applied beyond their valid span.

For recorded allocation or rebasing, use event-order mapping history, not merely the final captured map. For compressed children, retain the parent-member transformation rather than claiming a simple offset relationship. A recorded map is structural evidence; it is not a complete memory-content history.

3. Indirect calls, switches, and honest coverage

Direct branches encode enough information to recover a target easily. Indirect transfers obtain a destination through a register or memory. That may represent an import, callback, virtual method, jump table, or ordinary function pointer. The difficult part is following how the value was produced and whether it can change.

Use the analysis demo as your reference specimen

1. Save and choose **Load analysis demo** in Disassembly. Let Analyze code finish.
2. Find the dispatch routine and open **Indirect targets & switch tables**.
3. Inspect the guard before the dispatch. Determine the admitted input range and default path.
4. Expand the recovered table. For each of the three cases, record the label, entry VA/file offset, raw stored value, and resolved target.
5. Verify at least one entry in Hex, then follow its target in code. Compare the Blocks edges and Cross-references switch entries.

Absolute versus relative entries

An absolute table entry contains a target value directly. A signed-relative entry contains a displacement that must be added to the specified base. As a separate arithmetic example, a 32-bit entry `f0 ff ff ff` interpreted as signed little-endian is -16. With relative base 0x2000, the target is 0x1ff0. Treating it as unsigned gives a nonsense distant target.

The base is part of the compiler's scheme; it is not always the entry address or instruction PC. Check the actual address-building instructions. A guard using an unsigned comparison can bound a table; a nearby signed comparison is not interchangeable. Case labels can begin at a nonzero value after subtraction.

Supply a hypothesis only when you can defend it

Select the dispatch instruction, choose **Use selected instruction**, and enter a target plus a note, or supply table VA, count, width, encoding, relative base, and first label. Before saving, calculate one entry by hand and check that all proposed targets fit aligned mapped code. Tables are capped at 256 entries.

Save the hypothesis, inspect the new static edges, then remove it once to compare the result without your contribution. This teaches which information came from the file and which you supplied. Hypotheses can extend static discovery. They do not patch memory, install debugger hooks, or automatically change Ghidra's internal control flow.

What automatic propagation really knows

SPECTRA follows selected x86/x64 and AArch64 address-building sequences inside a basic block. Calls, joins, and unsupported operations discard facts. A read from a writable slot is a candidate because runtime writes can change it. Mixed ARM/Thumb transitions and other register-built targets can remain unresolved.

Increasing the instruction budget answers a coverage question, not a precision question. Continue analysis may discover more reachable bytes while leaving the same unresolved dispatch. Conversely, one justified target hypothesis can expose a function without decoding megabytes of unrelated content.

Report the boundary

Record the analysis budget, partial/finished queue status, unresolved transfer count, and any analyst edges. “No callers found” means no callers found by this pass. “Observed case 2 under input 2” is a runtime statement only after an actual modeled run.

Exercise: What evidence would distinguish a jump table from an array of ordinary data pointers? **Answer:** A bounded index, compatible entry interpretation, executable targets, and an instruction sequence that transfers control through the loaded value. A row of pointer-looking numbers alone is insufficient.

4. Recognize library work without erasing application logic

Library recognition saves time by separating common support routines from the behavior you came to study. It can also mislead if a familiar name gets attached too confidently. SPECTRA distinguishes declared imports, forwarding thunks, and exact-byte reference candidates. Preserve that distinction in your notes.

Three different kinds of evidence

- **Declared import:** the file asks for a named external symbol. That names an interface, not necessarily the runtime implementation it receives.
- **Import-forwarding thunk:** a small routine transfers through a recognized slot. Inspect whether it changes arguments or state before forwarding.
- **Exact function bytes:** the recovered contiguous bytes match a reviewed corpus entry. This is evidence about those bytes and boundaries, not proof of original source authorship.

The starter corpus contains 30 x86/x64 builds of small Church memory/string implementations. It is not a comprehensive libc, Windows, or firmware library database. Compiler versions, optimization, relocations, and boundary differences can defeat an exact match even when the high-level work is similar.

Case exercise

1. Load the analysis demo and open Functions.
2. Inspect library-function evidence for one candidate. Read the source/basis and any competing names.
3. Open its instructions. Identify its argument use, loop or copy/compare behavior, and return.
4. Use **Use this name** only when you accept the evidence. Check the resulting Notebook entry.
5. Return to a caller. Ask what the application passes to this routine, rather than spending all your time on the common helper.

Build a small reviewed signature pack

Select a well-understood contiguous function of 16-4096 bytes, give it a precise name, and choose **Export selected function signature**. Keep its source/build recipe and provenance with the pack. Import it into another investigation and inspect the match evidence. Current packs are bounded to 512 entries and 256 KiB of signature bytes; they do not run scripts or wildcard rules.

A pack should not silently turn “looks like memcpy” into an official library identity. Preserve ambiguity when two entries share bytes. Different names can compile to identical simple routines. A false name can contaminate later type and behavior assumptions more than leaving a generated address name.

Use absence properly

No match means the corpus did not establish a match under this recovery. It does not mean custom code, malware, or a novel algorithm. An import named memcpy can also be wrapped or replaced at runtime. Combine recognition with call-site evidence and the actual question.

Deliverable: one recognition decision with byte/boundary evidence, one caller explanation, and one sentence stating what the signature does not establish. Reconstruct affected code after a rename; stale output may still carry the old interpretation.

5. Recover structures and propagate useful corrections

Repeated loads from a base pointer at fixed offsets often suggest a record or object. A type turns those offsets into a working layout. It should explain the observed accesses, widths, alignment, and callers, not merely make the output pretty.

Case file: sixteen bytes with a layout

Open companion `record.bin`. Its exact bytes are:

```
53 50 45 43  03 00 00 00  10 00 00 00  00 00 00 00
magic        count        payload_size  reserved
```

This is synthetic teaching data. The field names are supplied by the exercise; in a real investigation they would begin as hypotheses. We will use little-endian 32-bit values for count, size, and reserved.

1. Open Decompile's **Structure workspace**. Select x86-64, little endian. Under Raw firmware mapping, enable a raw range with offset 0, length 16, load VA 0x1000, and function VA 0x1000 so the layout has a native language context. These bytes are data: do not decompile or execute them as instructions.
2. Set Structure name to `TrainingRecord` and Total byte size to 16.
3. Edit/remove the starter rows as needed, then add a char field at offset 0 named magic with count 4.
4. Add uint32_t fields at offsets 4, 8, and 12 named count, payload_size, and reserved, each with count 1.
5. Save structure. Set Preview at file offset to 0 and choose **Read fields from file**.
6. Expect magic's byte values 83, 80, 69, 67; count 3; payload_size 16; reserved 0. The preview may show numeric char values rather than spelling SPEC.

From layout to a function hypothesis

```
struct TrainingRecord {
    char magic[4];
    uint32_t count;
    uint32_t payload_size;
    uint32_t reserved;
};
```

If a routine reads 32 bits at pointer+4 and compares the value with a bound, this layout offers one explanation. To justify it, find callers constructing the same region, other field accesses, and where the alleged length is used. A count and a byte size can share the same integer type while expressing different units.

When applying a type to an actual native function, use **Tell Ghidra the types you know**, supply a matching declaration/signature, enable **Apply and save this function's assumptions**, and decompile again. Compare before/after output and inspect warnings. Saved declarations/signatures inform compatible callers in the same language profile; they do not apply across arbitrary CPUs or ABIs.

Do not conflate the correction channels

Address names follow supported views. Notebook signature hypotheses remain annotations until applied through the decompiler controls. Variable/type notes do not rename Ghidra locals. Static indirect-target hypotheses do not force Ghidra edges. Debugger patches do not modify the original file. These boundaries matter when two panels seem to disagree.

Packing, alignment, and pointer fields

The visual layout editor requires ordered, naturally aligned, non-overlapping fields within a size compatible with their alignment. Packed layouts, nested structures, and bitfields need manual C declarations where supported. The file preview does not dereference pointers. A plausible pointer value is not permission to read beyond the selected artifact.

Exercise: A field at offset 4 is loaded with a byte instruction, while your layout says uint32_t. What next? **Answer:** Inspect neighboring accesses and the consumer. It may read one byte of a larger field, or the hypothesis may be wrong. Do not “fix” the instruction interpretation solely to preserve your type.

6. Call a native routine with a controlled contract

A function harness supplies enough calling context to ask a small question without reproducing an entire application. It does not infer missing object state, initialize every global, or invent dependent services. First inspect a real caller: argument placement, widths, pointers, lengths, and any setup before the call.

Case file: choose a path

1. Choose **Load function + batch demo** in Debugger lab. It supplies x64 raw code at 0x1000 with System V x64 arguments 2 and 3.
2. Inspect the raw bytes and decompile the function. Its intended expression is `left ? left + right : right`.
3. Read the prepared argument registers and run to a clean return. Expect 5.
4. Reset and use Step instruction to observe the test and conditional jump. Locate where zero and nonzero inputs diverge.
5. Save a baseline checkpoint and the recipe before changing input.

The ABI form supports single machine-word integer/pointer arguments for System V x64, Windows x64, x86 cdecl/stdcall, ARM/Thumb AAPCS32, and AArch64 AAPCS64. Floating-point, aggregates, varargs, multiword values, and hidden object/result conventions are not inferred. RISC-V and MIPS currently require explicit register/stack setup.

Argument locations you should verify

Profile	Initial integer/pointer positions	Simple return
System V x64	RDI, RSI, RDX, RCX, R8, R9	RAX
Windows x64	RCX, RDX, R8, R9; reserved stack area	RAX
ARM/Thumb	R0-R3	R0
AArch64	X0-X7	X0
x86 cdecl/stdcall	Stack arguments	EAX

Consult the AMD64 psABI, Microsoft x64 rules, and AAPCS64 when an actual signature exceeds these simple examples. The form's supported subset is smaller than those standards.

Use a real buffer for a pointer

In Arguments, an entry such as `{"buffer":"input"}` supplies a mapped pointer, not the buffer contents as an integer. Put this in the two separate input fields:

```
Arguments:
[{"buffer":"input"},"3"]

Input / output buffers:
[{"name":"input","hex":"41424300","size":32}]
```

This creates ABC followed by a zero and padding. Use it only for a routine whose signature expects that pointer and length. It is not the right input for the arithmetic demo. The explicit size gives readable/writable capacity; it does not prove the routine will respect a supplied length.

Apply edits at the right time

Initialize uses the current setup editors. Reset repeats the last initialized setup. Register overrides apply after argument preparation and can replace a value you thought the ABI form supplied. Before a confusing run, inspect both the recipe and actual registers.

In Windows function-entry mode, the model retains mapped data and imports but skips execution of TLS callbacks and process/DLL entry. A function that expects their side effects needs additional justified state. Record that omission rather than attributing its fault to the original program.

Deliverable: a completed return for the demo, a documented branch address, and a caller-state checklist for one routine of your own. A faulting run can still inform the checklist; do not present its return register as a completed result.

7. Batches, watchpoints, checkpoints, and negative controls

One successful input can hide a wrong model. Use related cases that discriminate between explanations. A negative control is an input expected not to produce the effect. A boundary case tests the transition where behavior should change. Neither requires a huge random test set.

Run the prepared batch

The function demo prepares zero, two, and seven as the left argument, with right fixed at three. Choose **Run cases**. Expect clean returns 3, 5, and 10. The zero case uses the alternative path; the other two share the nonzero path. Inspect the observed locations/transitions rather than inferring path coverage from the result alone.

```
[
  {"label":"zero", "arguments":["0","3"]},
  {"label":"two", "arguments":["2","3"]},
  {"label":"seven", "arguments":["7","3"]}
]
```

Each case starts fresh. Interactive register edits, memory patches, breakpoints, and watchpoints are not inherited. A case replaces collections such as buffers or hooks with its override; register overrides merge by name. Read the resulting setup if you expected an inherited item to remain.

Interpret the report correctly

The baseline is the first case whose setup succeeded. A faulting baseline remains faulting, so added/missing locations relative to it can be misleading without stop reasons. A report's observed coverage is bounded; it is not a percentage of the whole binary or proof that unvisited paths are unreachable.

Select a case to inspect its buffers and modeled output. Clicking a coverage location can read bytes from the current interactive session, which may differ from that case. A batch report is stored evidence, not a live restored emulator. Use **Show interactive result** to make the context explicit.

Catch the first meaningful divergence

1. Predict a branch or output-buffer write that should differ between two inputs.
2. Prepare the first input interactively, initialize, and set a focused breakpoint/watchpoint.
3. Run to the stop, capture relevant state, and save a checkpoint.
4. Initialize the second input and repeat the same observation.

5. Compare the immediate causes: argument value, flag, loaded field, or modeled service result. Do not stop at “the final outputs differ.”

Replay is a test too

Save up to eight named checkpoints. **Restore by replay** executes recorded attempts/edits in a fresh emulator and verifies state. It requires the same sample and engine fingerprint. Compare & export reports registers, memory digests/permissions, and applicable model state. A checkpoint from another engine build is not made compatible by relabeling it.

Limits include 16 batch cases, up to 10,000 instructions per case, and 30 seconds overall. Pause/cancel should leave the previous complete report available; an incomplete attempt is not a new complete result. Trace and event retention are also bounded. Choose breakpoints to preserve the evidence you actually need.

Exercise: Two cases both return 5. Are their paths identical? **Answer:** Not necessarily. Inspect coverage, intermediate state, side effects, and stop reasons. Equal outputs can conceal different work or failures.

8. Verified history: reason backward within a bounded routine

Routine history is useful when the question concerns a specific state transition: which attempted instruction wrote a field, which analyst edit changed a branch, or what the buffer contained immediately before an access. The current mode is deliberately narrower than full process time travel: x86/x64, Bare routine, bounded mapped memory and a finite action log.

Construct a reproducible origin

Initialize from known sample bytes and a documented routine setup. Choose **Start history** before any step or register/memory edit. Include the ABI, entry, argument buffers and intended return boundary in your note. If the mode rejects the setup, use the ordinary debugger with a recording; a different CPU or process environment cannot become supported by changing a label.

Use the decoding demo as a controlled first experiment. Choose a 512-action budget, identify the output range, and predict the first store from the loaded byte and XOR key. Step until the store occurs. Record the action boundary, instruction address, before/after register values and output byte. Now inspect the immediately preceding boundary and compare your prediction with both states.

What makes a seek a check rather than a picture?

A seek reconstructs state in a fresh emulator from the initial state and recorded actions, then verifies the expected state digest. It does not just paint old register text over the current machine. Read verification failures as failures: do not cite a requested boundary as successfully reconstructed unless the seek completed. Replay can be expensive because each seek starts from the origin; periodic checkpoint acceleration is not implied.

Past state is read-only. **Return to latest** preserves the live experiment you left. Editing an arbitrary past point into a new branch is outside this mode. To compare a different input, create another documented experiment instead of describing a historical inspection as a new execution branch.

Action boundaries and faults

A timeline action can be an analyst edit or an instruction attempt. The action count is therefore not necessarily the count of successful CPU instructions. A failed attempt may have partial effects; retain the fault, boundary and state change as a unit. An edit is an input to the experiment, not a recovered fact about the original program.

Reverse to breakpoint / access searches recorded dynamic evidence. For a write query, verify that the returned boundary precedes the relevant write; inspect the next action to see its result. An absent access can mean it did not occur in the captured span, or that per-action

access capture was clipped. The limit is 128 captured accesses per action. Do not turn absent retained evidence into a proof that an instruction never accesses the region.

Design the smallest distinguishing test

1. State two competing explanations: the first store uses the current key, or it uses the updated key.
2. Choose an input for which those keys differ and where the output location is easy to identify.
3. Capture from initialization, step through the load/XOR/store/update sequence and inspect the boundary before the store.
4. Track both the input byte and key width. Confirm which value reaches the store; then inspect the key update separately.
5. Save a concise evidence note with boundaries and addresses. Include your controlled input and any supplied memory, not just the winning explanation.

This is a general method for stack strings, field initialization and small transformations. It is more informative than running until readable text appears, because it establishes a causal relation for a particular operation.

Persistence and capacity are part of the result

The maximum mapped memory is 2 MiB; action options reach 2,048 with a 4 MiB metadata bound. Budget exhaustion stops before the next action instead of evicting the beginning. The history remains session-local. Project/evidence export can preserve a selected state through supported checkpoint replay, but it does not serialize the complete routine-history timeline. Imported traces are not a substitute.

Windows/Linux process models, ARM-family, RISC-V and MIPS history remain outside this implementation. Persistent observation recording is a different mode with different capacity and restoration semantics. Stop or discard the active mode before starting the other; they do not capture simultaneously. Specify which mode produced each artifact in your handoff; “debugger trace” is too vague when one file contains observations and another represents a replayable state.

9. Trace provenance, persistent segments and event alignment

A trace answers a question only within its capture contract. Begin with image identity, environment, initial inputs, recorder/engine version, capture start, ordering, budget and completion state. These determine whether an apparent missing event is useful evidence or simply unobserved history.

Choose the artifact format deliberately

Artifact	Interpretation
Compact trace v1	Checksum-validated bounded observation export; latest 200 instruction records, bounded model channels and final mappings. Two read-only import slots. No emulator initialization.
Persistent recording v2	Ordered observations saved in incremental local segments, with mapping changes and explicit budgets. Incremental NDJSON export requires a completion footer.
Checkpoint / routine history	State reconstruction under the supported replay contract. It is not conferred on arbitrary imported observation files.

Capture a controlled pair

1. Choose a short benign routine or supported process exercise. Save the setup and initial input. Initialize, then **Start recording** before execution.
2. Use a small byte budget first; available choices are 2, 8, 32 and 64 MiB. The recorder also bounds instruction records at 100,000 and total records at 250,000.
3. Run to a meaningful stop, inspect partial/fault status and choose **Stop recording** to flush the final segment. Record whether the run completed or only capture stopped.
4. Export and save the completed file. Confirm the final completion status. Repeat with exactly one changed input, using a fresh initialization.
5. Open both for inspection. Compare engine/sample identities and supplied inputs before comparing their events. A changed image or profile is another independent variable.

Persistent segments preserve order across their boundaries; page navigation is presentation, not a new execution epoch. The viewer pages 100 rows at a time. A missing or interrupted tail must remain identifiable. Recovery of completed segments after a killed tab cannot manufacture the last unfinished one. IndexedDB is local persistence, not a durable archival guarantee.

From address lists to mapping history

A memory location can change meaning as mappings are added, removed or rebased. Inspect mapping-change records in event order and use the mapping applicable to the observation. The final map alone may not explain an earlier event. For file-backed regions, retain source offsets and sample identity; for allocated or zero-filled memory, retain the fact that no original byte exists.

Mapping history is not a complete memory snapshot at every event. A record that a region exists does not establish its entire byte content. Likewise, a file offset conversion does not prove that code actually executed at that VA. Keep structural, observed and reconstructed evidence separate in the report.

Read-only imports and external adapters

The supported external adapters include uncompressed x64dbg trace-file version 1 for trace32/trace64 and a bounded Chrome trace JSON import (8 MiB input cap). These formats provide different kinds of observations. An event-duration trace is not a register-by-register CPU recording. Unsupported variants should fail or retain explicit limitations; do not edit a format header to force acceptance.

Imported evidence retains its provenance and unverified status. A checksum is not an author signature and cannot validate a claimed execution environment. If the engine or sample differs from your open investigation, retain the mismatch. Address navigation is justified only with a matching artifact and mapping; the fact that an address lies within some open file is not enough.

Alignment is a proposal

Retained-order comparison aligns row order. Event-aligned comparison can accommodate a short extra path before a shared event, using bounded lookahead. Current alignment is limited to 24 lookahead records, 2,000 rows and 4 MiB of comparison data. Examine skipped/unmatched spans and distinguish a truncated comparison from a fully compared input.

Two equal event labels can hide different parameters, writes or model responses. Repeated events can produce more than one plausible match. Use aligned rows to formulate a candidate divergence, then inspect operands and earlier conditions. Do not report a semantic equivalence result from this UI.

Export failures are evidence-management events

A complete persistent-recording stream has its required footer. Interrupted output should remain visibly incomplete; an extension change or copied footer cannot repair missing records. When retrying, identify which saved recording is the source and which export is complete. After successful download, retain its digest and provenance alongside your report.

For a colleague's trace, a defensible sentence is: “This imported recording reports a write to this mapped region after the listed event; I have not independently replayed the sample.” For your own controlled experiment, specify the supplied input and model. Neither sentence needs to pretend the observation describes every possible execution.

10. Windows recipes: supply state without inventing reality

When a routine expects a clock, configuration file, or service response, an explicit model can isolate the part you want to study. The essential rule is to label what you supplied. A controlled response is evidence about the downstream code under that response, not about the real service's behavior.

Case file: the clock is an input

1. Load the x64 Windows demo and run its unmodified setup once. Save relevant API events and output.
2. Open **Environment recipe · mapped memory, API hooks, virtual files and ARM MMIO**.
3. Supply the recipe below, initialize again, and run the same entry.
4. Find the event marked as an analyst hook. Confirm the fixed result and compare downstream behavior with the baseline.

```
{
  "label":"Fixed clock for a repeatable experiment",
  "hooks":[{
    "api":"kernel32.dll!GetTickCount",
    "abi":"win64",
    "argc":0,
    "result":"1234",
    "label":"Analyst-supplied clock value"
  }]
}
```

This hook replaces the selected call boundary; it does not change Windows or establish real timing. The benign demo does not branch on the clock value, so identical final output is expected. That is a useful negative control: you can confirm the hook occurred without claiming an output effect that the code never expresses.

Virtual files and memory

A recipe file such as `{"path":"C:\\config.txt","hex":"746573740a"}` makes recorded bytes available under a modeled path. It does not read a same-named host file. Importing a local file into the virtual-input builder copies selected bytes into a recipe; editing the recipe requires Initialize to affect a new session.

```
{
  "regions":[{
    "address":"0x20000000", "size":4096,
    "permissions":"RW", "hex":"41424300",
    "name":"Captured input state"
  }]
}
```

That region is zero-padded after ABC and a terminator. Regions must respect page alignment and avoid existing mappings. Record the origin of captured RAM and whether it belongs to the same build/load address. Mapping zeroes until a fault disappears is not equivalent to reconstructing the missing object.

Output-producing hooks

A hook can write fixed bytes through a pointer argument and return a result. Its argument count, ABI, offsets, buffer capacity, and return convention must match the caller. A write of four bytes to argument zero should be documented as “recorded output supplied by analyst,” not “server response observed here.” Unknown APIs still stop if no supported model or explicit hook handles them.

Hooks skip the selected target body. They do not automatically construct nested objects, run constructors, preserve every side effect, or implement callbacks. Exact-address hooks and named API hooks are different targeting methods; confirm which one was applied in the event log.

Model sensitivity

Try two plausible responses only when the downstream question depends on them. If one response opens a branch and another closes it, report that dependency. Do not select only the response that makes your original hypothesis look right. Use failure and empty-output cases where the interface permits them.

Deliverable: a baseline and hooked recipe, the labeled call event, a statement about unchanged/changed output, and a list of omitted real API effects. Keep secrets out of shared recipes and project exports.

11. Linux experiments: descriptors, bytes, and missing services

The small Linux model is useful when your question fits its static-loader and syscall subset. Think in terms of byte buffers and explicit state. It does not create a shell, sockets, threads, signals, arbitrary shared libraries, or a real /proc filesystem.

Extend the echo experiment

1. Load the x64 Linux demo, record its read/write/exit sequence, and save a baseline.
2. Set stdin to empty and initialize. Predict an empty output and zero exit. Run and verify the returned read count.
3. Set stdin to `4142ff0a`. Run and inspect the raw output hex, not only displayed text.
4. Supply more than 32 bytes and predict the single read's maximum contribution to output. Check the actual count and bytes.

The byte 0xff is not valid as a standalone UTF-8 character, so a text preview may show a replacement character. That does not mean the model changed the raw output bytes. Reports retain separate stdout/stderr hex and combined emission-order bytes. Preserve the representation suited to the claim.

Process state is data

```
{
  "argv":["sample","--inspect"],
  "env":["MODE=training"],
  "stdin":"68690a",
  "files":[{"path":"/config","hex":"41420a"}]
}
```

Those arrays and buffers become modeled startup state. They do not invoke a command named sample. The simple echo fixture ignores argv/env/config; another program might read them. State which input a target actually consumes before attributing a behavior to a recipe change.

Descriptors and file position

A read consumes bytes from the current descriptor position. Repeating it can produce different results or EOF. Closing a descriptor changes later behavior. A modeled error such as a bad descriptor is different from an unsupported syscall. Read the numeric result, model note, and arguments together.

Virtual regular files and supported calls are bounded. Current individual-file and I/O limits are 64 KiB, with additional total-file/output limits. open/openat support selected variants; anonymous memory mapping, brk, protection changes, and other calls have explicit admitted

arguments. A recognized syscall number does not guarantee every flag combination is modeled.

Memory protection is part of the experiment

A buffer can exist but be unwritable. A range can cross a mapping boundary. A requested fixed mapping may be rejected because it overlaps image, stack, or another protected region. These failures can distinguish a wrong setup from a legitimate program request the model does not cover.

Some failed operations can be partial. Inspect memory writes and the trace's partial marker before assuming no side effect occurred. SPECTRA excludes partial operations from observed coverage; absence from that collection does not erase the recorded partial state change.

When you hit the boundary

If the next operation requires a socket, a thread, or dynamic linking, record the missing requirement. You can often isolate a downstream parser with captured data or study the call statically. Do not rename the environment or treat a missing service as a harmless error unless the actual caller's error handling supports that path.

Exercise: The text output looks corrupt but exported hex matches stdin byte-for-byte. Where is the discrepancy? **Answer:** In the text interpretation/display, not necessarily the byte transformation. Keep raw bytes as the primary output evidence.

12. ARM MMIO and portable hardware reasoning

Firmware often reads an address that stands for a peripheral register rather than ordinary RAM. That is memory-mapped I/O, or MMIO. A read may acknowledge an event, consume a FIFO entry, or expose a changing status bit. Supplying a fixed RAM page can therefore misrepresent the device.

Case file: read 41, write 42

1. Save and choose **Load ARM register demo**. The ARM routine reads a word through R0, increments it, writes it back, and returns.
2. Inspect the function harness: AAPCS32 with argument 0x20000000.
3. Inspect the MMIO recipe: width 4, one supplied read value 41, and captured writes.
4. Run to return, then open **Recipe, Linux and hardware events**.
5. Expect one supplied read of 41 and a captured write of 42. The bytes/instructions explain why.
6. Run the prepared cases for 41 and 99. Expect captured writes 42 and 100 respectively.

```
{
  "mmio":[{
    "address":"0x20000000", "width":4,
    "values":["0","1"],
    "onExhaustion":"stop", "write":"capture",
    "label":"Recorded status sequence"
  }]
}
```

This second recipe illustrates a two-read sequence. The demo itself reads only once, so it would consume 0 and write 1; it does not demonstrate a polling loop. To study a real loop, use a routine whose instructions actually repeat the read and record why the chosen sequence represents a useful device scenario.

Why exhaustion matters

After the supplied values are consumed, stop is an honest boundary. Explicit hold repeats the final value and can sustain a polling loop, but that is another assumption. Captured writes do not automatically alter future read values or trigger peripheral behavior. A write of 42 is a recorded event, not proof that a simulated device now contains 42.

The model admits exact aligned 1-, 2-, or 4-byte accesses for ARM/Thumb/AArch64. Wrong widths, unknown addresses, forbidden writes, or exhausted sequences stop. MMIO pages cannot overlap ordinary mappings and this recipe mode is separate from the Linux process profile.

Other CPUs

The Training ground hardware exercise also offers RISC-V and MIPS. RISC-V's supplied compressed-instruction addition returns 5 for A0=2 and A1=3. The MIPS fixture returns 6 because its return delay slot adds one. These are tested integer routine profiles, not all ISA extensions or devices.

A MIPS branch/delay-slot pair is handled as an atomic supported step here. A breakpoint on the delay slot sees pre-pair state before the pair executes. If your mental model assumes every Step instruction corresponds to one displayed row, this detail can make the trace seem wrong.

RISC-V ABI distinctions and extensions matter; consult the RISC-V psABI specification when selecting a real target. An ELF metadata mismatch is a reason to investigate, not reflexively accept the override. Likewise, microMIPS, MIPS16, R6, n32, ARM BE8, and arbitrary FP/SIMD behavior are not covered by selecting an ordinary profile.

Missing board behavior

No timing, DMA, interrupt controller, reset sequence, or complete peripheral state machine is supplied by a list of register responses. Use that list to isolate a computation or decision. Keep captured RAM separate from MMIO and tie both to the same build/address assumptions.

Deliverable: a routine-level explanation, read sequence provenance, width/address evidence, captured writes, and a clear statement of which device effects remain unmodeled.

13. Recovery as a chain of evidence

A recovered string is most useful when you can reproduce every step from original artifact to consumer. Preserve source range, representation, transform, parameters, output hash, and code evidence. A collection of plaintext screenshots loses the hard part.

Case file: one known XOR layer

1. Open xor-message.bin and inspect all five bytes in Hex.
2. In Recovery, select original offset 0, length 5, Repeating XOR, key 5a.
3. Decode and record the output bytes `68 65 6c 6c 6f`, its length, and hash.
4. Write the byte relation for the first position: 0x32 XOR 0x5a = 0x68.
5. Repeat the decode with a deliberately wrong key and compare the whole result. Treat this as a negative control, not a search for a lucky word.

For a repeating multi-byte key, key index depends on where the selected input begins. If code begins decoding at byte N but your range begins at N+1, the key phase may be wrong. An apparently correct algorithm can still fail because of range and phase.

Stack writes and memory captures

Static stack-string recovery recognizes certain constant-write sequences; inspect each contributing instruction and write width. Little-endian immediate stores can make the constant's printed number look reversed relative to text. Overlapping writes or intervening unsupported transformations can break a simple reconstruction.

An emulated capture has different evidence: supplied entry state, completed/partial operations, capture address, timing, and model assumptions. Compare capture length with the routine's produced length. Zero padding or stale buffer tails can create misleading strings if you capture much more than was written.

Cryptographic parameters are not decoration

SPECTRA's AES-CBC path expects an actual key, a 16-byte IV, and the admitted padded layout. AES-GCM expects a key, a 12-byte nonce, optional AAD, and ciphertext with a trailing 16-byte tag. Authentication failures can reflect wrong bytes, wrong layout, wrong AAD, or wrong key. They are not instructions to discard verification. Browser cryptographic APIs document parameter/error behavior in SubtleCrypto.decrypt; this lab exposes its own narrower controls.

Distinguish key material from a password or encoded string. A program may derive a key before use; copying its visible password text into the key box does not reproduce that derivation. Likewise, RC4 variants that discard an initial keystream prefix do not match a transform beginning at position zero.

Compression and nested layers

Verify gzip versus zlib versus raw DEFLATE framing. A decompressed result might be a configuration record, executable, or another compressed member. Use as next layer only when the resulting header/code supports that choice. Expansion is bounded to prevent one small input from consuming arbitrary memory.

For each layer, keep its own coordinate system. The record “layer 2 offset 0x20” needs a link back to layer 1 and the original source range. A transformed byte may have no single meaningful original offset. If you open it as a child investigation, preserve the parent/layer hash and chain in Notebook.

Consumer verification

The final question is “who uses the recovered data?” Follow the output pointer, length, or returned object into its next call. A string resembling a hostname may be diagnostic text; a decoded key may be unused test data. A coherent decode plus a meaningful consumer is stronger than either alone.

Deliverable: a transform ledger with input/output hashes, exact ranges, parameter sources, consumer, and unresolved steps. Exported recovery reports omit manually supplied keys, so preserve required reproduction information intentionally and privately.

14. Managed clients: methods, labels, and a useful harness plan

A thick client can spread one operation across UI code, session state, serializers, cryptography, and a transport. Your job is to identify a tractable boundary. Method labels help prioritize; callers, arguments, and state establish the actual behavior.

Case file: map LabClient

1. Load the .NET demo, browse types, and locate LabClient.
2. Read constructors, SetPrefix, and Greet. Draw the state transition in a note: default prefix ready; SetPrefix replaces it; Greet consumes it.
3. Read Fingerprint and identify the referenced hashing operation. Do not label hashing as reversible encryption.
4. Read Json and note the serializer boundary. Read FetchAsync and trace the supplied URL into HttpClient.
5. Inspect Credentials and MisleadingNameAuth. Compare actual referenced operations with the Auth label or name clue.

Find a real client's request builder by following an operation from its caller toward serialization and transport. Record field sources, encoding, route construction, headers, authentication state, and error paths. A URL string plus an HttpClient reference does not reconstruct a server protocol, prove a successful login, or reveal the server's validation rules.

Resolve metadata dependencies deliberately

Add local assemblies that belong to the same target build. Resolution uses full identity rather than filename alone. If a method body improves after adding the right dependency, record which artifact supplied that metadata. A duplicate identity is rejected; renaming a DLL does not change its assembly identity.

Source reconstruction does not execute dependencies. A downloaded harness is different: invoking target code outside the browser can execute constructors, static initialization, native interop, and other side effects. This manual's browser exercises stop at preparing and reviewing that artifact.

Select the smallest useful harness

1. Select the constructors/setup methods and Greet that you need, rather than Add entire assembly by default.
2. Choose Greet as First method. Select a runner SDK/CPU compatible with the intended target; a dropdown cannot convert a framework-specific application into a different runtime.

3. Inspect the generated starter recipe and exact method tokens for this assembly. Do not copy tokens from another build.
4. Represent default construction, SetPrefix("hello"), then Greet("Ada") as the desired invocation order. Predict hello:Ada.
5. Download the harness ZIP and export selection + clues. Read its source, argument representation, and compatibility warnings as data.

A setup recipe can run a bounded sequence on the selected instance, and LabHooks.cs is the extension point for custom dependency/session objects. Supplying a constructor and setup call is not the same as replaying an application's entire initialization. Generics, by-reference parameters, unsupported objects, or native dependencies can require work beyond the generated driver.

Proxy and session expectations

Optional HTTP proxy settings help only when the client honors those defaults. They do not guarantee interception of custom sockets, all handlers, or pinned connections. TLS validation remains enabled. Do not teach bypassing validation as a routine setup fix; document the transport boundary and use an authorized controlled service when an external test is eventually appropriate.

The runner previews until explicit execution is enabled, but real execution then has host permissions. A future browser-managed runner would need its own capability model, dependency limits, deterministic fixtures, and tests. The current downloadable harness should be described honestly as preparation for an external lab.

Deliverable: a minimal method selection, dependency identities/hashes, ordered state recipe, predicted return, label evidence, and a list of transport/runtime questions that remain untested.

15. Android: connect package, DEX, source and native evidence

Android analysis crosses several coordinate systems and semantic layers. Treat the APK as a container; each DEX as its own program metadata and bytecode; reconstructed Java as an engine interpretation; resources as a separate data model; and native libraries as architecture-specific binaries. A single application name does not collapse these distinctions.

Build an identity ledger before reading classes

Load the Android teaching fixture or an authorized APK in **Decompile → dex2jar + Vineflower → Read APK / DEX**. Record APK SHA-256, DEX filenames and hashes, selected class identity, method descriptor and original code offset. Standard root multidex members are indexed independently. Duplicate identities remain visible: select the intended DEX rather than merging two definitions mentally.

Source navigation crosses unambiguous definitions in the package. In the teaching fixture, decompile SecondDex.total and follow the reported reference to Training.sum in classes.dex. Inspect both signatures and the argument/return relation. Keep the method's DEX hash with any copied snippet; a Java filename alone is insufficient provenance when classes are renamed or duplicated.

Audit the reconstruction chain

The pipeline translates DEX to JVM class bytes with dex2jar, then uses Vineflower to reconstruct Java. Engine warnings belong in the finding. Missing Android framework or external metadata can affect types, casts and inheritance; class conversion failure is different from a conclusion that the original code is invalid. Kotlin input still yields Java on this route.

Use **Inspect method bytes** for the original DEX descriptor, code location and instruction length. The preview is bounded to 256 bytes. Source tokens and references do not establish exact line-to-instruction mapping. When an interpretation matters, identify the original method and relevant bytecode evidence; do not label a native ARM decode of DEX data as supporting disassembly.

Challenge the obvious story

The fixture declares INTERNET, contains strings and performs arithmetic. Those facts do not imply network activity. Construct a claim ledger with four rows: declared capability, referenced operation, reachable path under stated assumptions, and observed behavior. A row may remain empty. This prevents permissions and suggestive method names from becoming a fictional behavior report.

For a real client, trace data into an endpoint consumer, serializer or crypto API without contacting the endpoint. Record whether a path is direct, reflective, native or unresolved.

Dynamically loaded code absent from the APK remains missing evidence. A failed source reference may indicate an external dependency, ambiguity or reflection; it does not prove the target cannot exist at runtime.

Resources are evidence, not a rebuilt app

Decode the manifest and resources.arsc through **Manifest, resources & package files**. Preserve numeric IDs, key/type names, scalar or bag values and raw variant configuration where available. The current view does not resolve every framework default, overlay or device selection. A variant with an interesting URL does not establish that a given device selects it.

Resource-only splits may still be inspected, but split-set merging is not supplied. APK signatures and publisher authenticity are not verified. ZIP CRC and DEX integrity diagnostics serve a different purpose from provenance or publisher trust. A successfully decoded table is not an authenticity check.

Follow native children without losing the parent

1. Save the APK investigation and inventory before selecting a native member under `lib/<abi>/`.
2. Use **Open child investigation**. Check the ELF machine and format rather than trusting only the directory spelling.
3. Inspect its native functions with the appropriate Ghidra language and ABI. Keep any linkage to managed declarations as an evidenced hypothesis; native registration can be dynamic.
4. Record parent hash, member path, compression/extraction context and child hash. A decompressed child offset is not generally a linear offset in the original APK.
5. Restore the parent investigation when returning to managed code. Use separate saved investigations or a collection for each layer.

Know which limits affect your conclusion

The Android path accepts standard DEX 035 and 037-040, with up to 32 DEX files, 32 MiB per DEX and 64 MiB combined. Top-level input is bounded at 64 MiB and combined classes at 50,000. Supporting conversion and source output have their own limits. These are work bounds, not a statement of total browser memory consumption.

Compact/optimized DEX, DEX 041 containers, packed or downloaded code and Android runtime debugging require other support. Record an unsupported format as unsupported; do not generalize the failure to all Android applications. Cancel terminates the worker, preserving the original file for a later fresh operation.

Deliver a connected result

Export selected source plus evidence and inventory. Include engine identity, warnings, DEX hash/descriptor, the manifest/resource facts used and the native-child provenance if relevant. A selected reconstructed class can persist in workspace/projects and analysis evidence; this does not mean every class was decompiled. Finish with a narrow finding and the next missing dependency or path to investigate.

16. Firmware and patch review as connected investigations

A firmware update can change an outer archive, one library, a configuration default, and a parser at the same time. Compare at the right layer. A different compressed package hash tells you the download changed; it does not identify the behavioral change.

Extract with provenance

1. Save the old and new parent investigations with both hashes and vendor/source context.
2. Inspect each supported container at a justified offset and export inventories.
3. Match candidate members by path, hash, size, and identity. A matching path alone can hide a completely different component.
4. Open relevant regular members as child investigations, keeping parent hashes and extraction metadata.
5. Compare the two child binaries in Binary diff. Preserve unmatched and ambiguous groups.

Compressed offsets, inode metadata locations, and decompressed member offsets have different meanings. SPECTRA's inventory is not a verified package SBOM. A library filename/version string is useful evidence, but backported fixes and vendor changes can make version-only conclusions wrong.

Practice with the supplied PE pair

Compare add-one.exe with add-two.exe and explain the immediate change at entry. Decompile the pair, then call each entry as a Windows x64 native function with input 7. Expect 8 and 9 after clean returns. The surrounding strings and imports remain fixture data, showing why unchanged metadata does not imply unchanged code.

Now imagine a real change from a copy operation to a guarded copy. List the evidence required before claiming a fix: the new condition, its signedness and units, where length originates, the guarded operation, failure behavior, and which callers can still reach it. This is an analysis worksheet, not an exploitable target or proof that a particular product is vulnerable.

Candidate matching is a hypothesis

Inlining, optimization, symbol stripping, register allocation, and layout changes can destroy a simple one-to-one match. A shape match can align routines with different purposes. An exact decoded-byte match can still call changed dependencies or read changed data. Review both the match basis and the function boundaries.

When raw mapping is active, applying the same offsets to both files is justified only if their layouts agree. A shifted firmware region needs separate mapping work. Otherwise you may compare old code against new padding and interpret the mismatch as a dramatic rewrite.

Test around the changed condition

For an authorized benign parser or reproducible routine, choose inputs just below, at, and above the relevant bound, plus a normal case and a rejected-format case. Keep the same ABI/model and compare return status, buffers, and paths. If the model cannot supply a dependency, report the static conclusion separately from unperformed runtime validation.

Deliverable: parent-child provenance, candidate function mapping, exact changed logic, test inputs/results if supported, and the limited conclusion. “This observed path now checks X before Y” is stronger and narrower than “the firmware is secure.”

17. The intermediate casebook: contracts, counterexamples and handoffs

The next five campaigns remove some of the beginner scaffolding. Ghost Circuit connects calling conventions, structures, dispatch and state machines. Hollow Signal connects a rolling decoder to a bounded parser and a guard comparison. Iron Veil connects name hashing, collisions, indirect calls, path constraints and coverage. Black Loom adds reconstruction, rebasing and partial-result analysis. Puppet Core adds custom instruction semantics, virtual state and interpreted control flow. All use small benign artifacts; no stage requires contacting a service or running a specimen natively.

Carry four ledgers

Ledger	Keep these facts
Identity and map	Artifact hash, file offset, VA, region, width, endianness and the supplied or inferred mapping rule.
Value and contract	Argument register, pointer/value distinction, field width, transform order, return convention and its source.
Path and observation	Predicate, taken edge, skipped checks, experiment input, stop state and whether evidence is static or recorded.
Claim and counterexample	Hypothesis, strongest support, smallest input that distinguishes a competing model, and remaining uncertainty.

Use the supplied MEMORY-MAP, FORMAT-NOTES and worksheets as contracts for these fixtures. Do not generalize a supplied field name or ABI to every binary. In professional work those contracts may be missing; this training makes the consequences of an assumption visible before asking you to infer it from scratch.

Make the decompiler earn its place

Start with the exact mapped assembly. Reconstructed C can help organize branches, but cross-check widths, signed comparisons, indirect targets and call effects. When applying a recovered structure or signature, label it as an analyst hypothesis until code and data support it. A cleaner-looking listing is not a validation result.

Use SPECTRA's raw byte comparison for data and flat images, then decode each version separately with the same justified map. A second comparison file is not automatically the

main disassembly source. Save notes before switching ranges; the raw-range result is replaced by the next decode.

Worksheets without a second machine

Hollow Signal's browser Local trace worksheet evaluates a small rolling-XOR recurrence after you supply recovered parameters. Iron Veil's Name hash worksheet evaluates the recovered short hash. They are deterministic arithmetic helpers for the supplied small data, not native execution, a full emulator, or SPECTRA portable-trace imports.

Derive at least the first two iterations yourself, then use the helper to avoid transcription mistakes on the remaining bytes. Record parameters and input hash. A helper can consistently compute the wrong model if you enter the wrong rotate width or include a terminator the routine excludes.

A professional finish

Complete the public case worksheet and preserve your evidence. If using the Church Discord checker, submit privately through `/spectra` and the selected campaign. The checker scores the formatted answer; your research deliverable is the map, reasoning, distinguishing test and limited conclusion. You should be able to hand it to another analyst without asking them to trust the flag checker.

18. Ghost Circuit: one image, connected reasoning

Five linked routines share a record and pointer table; a separate state-step routine teaches model validation. The challenge is keeping values, widths and contracts intact across calls. Read MEMORY-MAP.txt first and decode only the listed CODE rows. All arithmetic and control-flow reasoning can be performed statically.

Open the public campaign casebook. Exact public practice files and hashes are in `practice/ctf/004/`. Save your current investigation before changing the main file.

Stage	Skill to carry away
01 · The argument trail	Windows x64 ABI + call sites
02 · Recover the record	Structure offsets + load widths
03 · Resolve the missing edge	Bitfields + indirect function calls
04 · Keep the width	Bit rotation + byte overflow
05 · Across the calls	Volatile registers + saved data flow
06 · Draw the machine	State transitions + reset behavior
07 · Break your hypothesis	Counterexamples + model validation

Keep the flat image mapped

```
VA = 0x6000 + file offset (this fixture only)
dispatcher: offset 0x000, length 0x43, VA 0x6000
field helper: offset 0x080, length 0x07, VA 0x6080
selector: offset 0x090, length 0x0b, VA 0x6090
state step: offset 0x0e0, length 0x3a, VA 0x60e0
record: offset 0x140; table: 0x160; events: 0x180
```

For other targets, use the exact CODE row in MEMORY-MAP.txt. All ranges use x86-64, little endian. Each Decode range replaces the previous code results; your saved notes preserve the relationship.

Stage 01 · The argument trail

The question. Begin with the dispatcher. At its FIRST direct CALL, identify the register explicitly written for the second integer argument and its value. Submit the lowercase 32-bit register name plus four hex digits. Learn to read a call site before guessing what the callee does.

Open these files: ghost-circuit.img.bin, MEMORY-MAP.txt.

1. Open ghost-circuit.img.bin and MEMORY-MAP.txt. Decode the dispatcher using its map row; select All decoded instructions. Keep the file open for this mission.
2. Windows x64 integer/pointer arguments use RCX, RDX, R8, R9. Find the MOV before the first CALL. LEA supplies a pointer as argument one; MOV supplies the integer as argument two.
3. Record the first call target, both argument meanings and the expected return register in Notebook. The map explains shadow space; it is not another input record.

Read what happened

The map describes a 512-byte flat image at VA 0x6000, not a PE file. Decode the dispatcher row, length 0x43, and distinguish LEA's address calculation from a loaded value. Windows x64 places the first two integer/pointer arguments in RCX and RDX. An instruction writing EDX supplies the low 32-bit integer argument; it is not a second pointer unless the code uses it that way.

At the first call, write a contract row containing call-site VA, target, RCX's pointer meaning, the immediate in EDX and EAX's return role. The 32-byte shadow space required by this ABI is caller-reserved stack space, not an extra argument record to decode. This case's convention is supplied; a raw x64 file does not inherently prove Windows ABI.

Hints, one at a time

1. This is the Windows x64 ABI, not the Linux SysV ABI. RDX is argument two; EDX is its low 32-bit register.
2. The first CALL is at VA 0x6010. Look immediately above it for the MOV to a 32-bit register.
3. Use the register spelling shown in that MOV, then pad its immediate to four hex digits. LEA RCX loads an address; it does not read the bytes at that address.

Response format: `SPECTRA{register_hhhh}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Keep the evidence: write the exact source offset or instruction, the operation you applied, and a sentence explaining why it answers this stage. Keep intermediate values labeled so the next stage does not confuse an input with a return value.

Stage 02 · Recover the record

The question. Decode sub_6080 and infer the field it reads from the record. Recover that unsigned field value BEFORE adding argument two. Submit four lowercase hex digits. Build a small record-layout table from actual load instructions, rather than guessing from strings.

Open these files: ghost-circuit.img.bin, MEMORY-MAP.txt.

1. Decode the sub_6080 map row. MOVZX reads a WORD from RCX+2: a WORD is two bytes, then zero-extended into EAX. ADD EAX,EDX is a later calculation.
2. RCX points to VA 0x6140. Convert the field VA to a file offset with this image's map, then inspect two little-endian bytes in Hex explorer. Do not read a DWORD or include the following field.
3. Write the offset, width and recovered value in your worksheet. Also compute and SAVE the first call's returned EAX after adding argument two; stage 05 will use it.

Read what happened

The helper at 0x6080 reads a WORD at RCX+2 with MOVZX. The load width is two bytes even though EAX is a 32-bit destination. Map the record pointer to file offset 0x140 and inspect its field at +2. Interpret the two bytes little endian before performing the later ADD with EDX.

Keep two names in your ledger: loaded_field and first_helper_return. They are different quantities. The stage asks for the field; a later stage needs the returned sum. A structure sketch should record offset, width and observed use without guessing a business meaning that the fixture did not supply.

Hints, one at a time

1. The field starts two bytes after the record. The record starts at file offset 0x140, so inspect file offset 0x142.
2. The bytes at that field are 2c 01. For a little-endian WORD, the second byte contributes the higher eight bits.
3. The requested flag is the loaded field, not the later sum. Keep both values in your notes and label them clearly.

Response format: `SPECTRA{field_hhhh}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Keep the evidence: write the exact source offset or instruction, the operation you applied, and a sentence explaining why it answers this stage. Keep intermediate values labeled so the next stage does not confuse an input with a return value.

Stage 03 · Resolve the missing edge

The question. The dispatcher eventually uses CALL R11 instead of a literal destination. Resolve that function pointer for the supplied record. Submit the target VA as four lowercase hex digits. Distinguish the table slot's address from the function address stored inside it.

Open these files: ghost-circuit.img.bin, MEMORY-MAP.txt.

1. Decode sub_6090. Read its BYTE field from the record, apply AND and SHR in order, and write down the returned index. The mask selects only two bits.
2. Return to the dispatcher. R10 is the table base; the MOV into R11 reads [R10 + RAX*8]. Compute the slot VA and convert it to a file offset. The table contains four 8-byte absolute VAs.
3. In Hex explorer, read that slot as one little-endian 64-bit pointer. Locate the resulting CODE row in the map. Record the manually recovered call edge in Notebook; an indirect edge can be missing from an automatic graph.

Read what happened

Decode the selector at 0x6090. The AND retains a two-bit field and the shift moves it down to an index between zero and three. Return to the dispatcher and calculate R10 plus index times eight. Eight is the pointer width, not the number of bytes in an instruction or a DWORD field.

The calculated slot address tells you where to read. The eight stored bytes tell you which VA to call. Those are different addresses. Map the selected target back into a CODE row and record the edge, its source field and the table slot as evidence. If automatic cross-references do not resolve this edge, that gap does not make it unreachable; your manual derivation should remain visibly tied to its input assumptions.

Hints, one at a time

1. sub_6090 reads record byte +1, masks with 0x0c, then shifts right by 2. Masking first keeps the index between 0 and 3.
2. The table base is VA 0x6160. Multiply the index by EIGHT, not four; these are 64-bit pointers.
3. Read eight bytes AT the computed slot. Submit the decoded pointer VALUE, not the slot VA, record address or file offset.

Response format: `SPECTRA{target_hhhh}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Keep the evidence: write the exact source offset or instruction, the operation you applied, and a sentence explaining why it answers this stage. Keep intermediate values labeled so the next stage does not confuse an input with a return value.

Stage 04 · Keep the width

The question. Decode the target recovered in stage 03. Trace its return value for the supplied record. ROR rotates bits; ADD AL wraps in eight bits. Submit the final EAX as two lowercase hex digits. This byte transform is obfuscation, not encryption.

Open these files: ghost-circuit.img.bin, MEMORY-MAP.txt.

1. Use the target's CODE map row. Identify which record byte MOVZX reads and inspect that byte in Hex explorer. AL is only the low eight bits of EAX.

2. For ROR AL,3, move the bottom three bits to the top and shift the others right. Write an eight-bit diagram. Then add the immediate to AL, keeping only the low eight result bits.
3. The final MOVZX EAX,AL clears the upper bits. Save this return value for stage 05. Optional check: a right rotation undone by a left rotation of the same width restores the original byte.

Read what happened

Read only the byte selected by the target routine's MOVZX. ROR AL,3 rotates eight bits; it is not a shift that discards bits and not a 32-bit rotation of EAX. Draw the original bit positions, move the low three bits to the top, then apply the byte addition and discard carry beyond bit 7.

The final MOVZX exposes the low-byte result as a zero-extended return. Record intermediate values and the width at every step. A calculator using arbitrary-width integers needs explicit masks to reproduce this operation. The exercise generalizes to obfuscated arithmetic and compact checksums: operation order and truncation are part of the algorithm.

Hints, one at a time

1. The selected routine reads record byte +4. A byte load sees only the first byte of the four-byte field there.
2. For eight-bit x: ror8(x,3) = ((x >> 3) | (x << 5)) & 0xff. The input byte here is 0xd6.
3. The rotated byte is 0xda. Add 0x29, discard any carry beyond eight bits, and zero-extend the remaining byte.

Response format: `SPECTRA{byte_xx}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Keep the evidence: write the exact source offset or instruction, the operation you applied, and a sentence explaining why it answers this stage. Keep intermediate values labeled so the next stage does not confuse an input with a return value.

Stage 05 · Across the calls

The question. Return to the dispatcher and calculate its final EAX. Explain why it saved an earlier return value on the stack. Submit four lowercase hex digits. The selected transform's return is only one part of the final result.

Open these files: ghost-circuit.img.bin, MEMORY-MAP.txt.

1. Trace the dispatcher from its first CALL to RET. Label each EAX value: first helper result, selector index, selected transform result, final sum.
2. At VA 0x6015 the first result is stored at caller RSP+0x20. Later calls may overwrite EAX; do not assume its old value survives. The final ADD reads the saved DWORD.
3. Combine your stage 02 returned sum with your stage 04 byte result. Record a short function summary with inputs, output and the evidence addresses. RCX is reloaded before each call that needs the record pointer.

Read what happened

EAX is reused across calls. Label its value at each return instead of treating the register as one permanent variable. The dispatcher saves the first helper's return at caller RSP+0x20 before making further calls. The selector's return is an index used for dispatch; it is not an extra arithmetic term in the final result.

Track the saved stack value through the selected transform's return and final addition. RCX is reloaded when a new call needs the record pointer. Finish with a high-level summary whose terms correspond to the loaded field, supplied integer argument and transformed record byte. This is dataflow across calls, including where a volatile register's old value survives.

Hints, one at a time

1. There are three calls. The index returned by the middle call is for dispatch, not an extra term in the final sum.
2. First helper: the stage 02 field plus the second argument from stage 01. That result is saved at RSP+0x20 before EAX is reused.
3. The final result is saved_first_return + selected_transform_return. The stack allocation is released after this addition and does not change EAX.

Response format: `SPECTRA{result_hhhh}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Keep the evidence: write the exact source offset or instruction, the operation you applied, and a sentence explaining why it answers this stage. Keep intermediate values labeled so the next stage does not confuse an input with a return value.

Stage 06 · Draw the machine

The question. Now investigate the SEPARATE sub_60e0 routine. ECX is the previous state; DL is one event byte; EAX becomes the next state. Start at state 0 and process the six event bytes at file offset 0x180 once each. Submit all six resulting states as a digit sequence, excluding the initial 0.

Open these files: ghost-circuit.img.bin, MEMORY-MAP.txt.

1. Decode sub_60e0 using its map row. Follow comparisons and branches; build a table for prior states 0, 1, 2, 3 and invalid states against A, B, C and other input.
2. Read the six bytes at file offset 0x180. For each byte, supply the LAST returned state as the next ECX. Record before/event/after rows. A mismatch consumes its byte; never retry it without an instruction to do so.
3. Describe state 3 and reset behavior in words. This routine is not called by the dispatcher: analyze it as a separate exercise. If the graph feels busy, follow one prior-state path at a time.

Read what happened

Treat sub_60e0 as a separate routine with ECX prior state, DL event and EAX next state. Build the transition table directly from comparisons and branches. For states 0, 1 and 2, the expected next character advances the machine; a mismatch resets it. State 3 is terminal and remains 3. Other invalid u32 states return zero.

Read the six event bytes at file offset 0x180 and make exactly six rows. Feed each returned state into the next call. A mismatch consumes its event; there is no implied retry. This subtle distinction separates a literal implementation from a friendlier pattern matcher you might accidentally imagine. The dispatcher does not call this routine, so report it as a separate analyzed function.

Hints, one at a time

1. Read 41, 42 and 43 as ASCII A, B and C. The event sequence is AXBABC. EAX starts at zero on each call unless a path assigns another return.
2. The forward transitions are 0+A→1, 1+B→2 and 2+C→3. Mismatches reset to zero. Prior state 3 always returns 3; other invalid u32 states return zero.
3. Make six rows, not seven. After the X resets the machine, the following B cannot advance state zero. Then continue through the remaining three bytes.

Response format: `SPECTRA{states_dddddd}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Keep the evidence: write the exact source offset or instruction, the operation you applied, and a sentence explaining why it answers this stage. Keep intermediate values labeled so the next stage does not confuse an input with a return value.

Stage 07 · Break your hypothesis

The question. Compare sub_60e0 with the three pseudocode models in 07-models.txt. Choose the exact model, then test the tempting alternative A using input AABC. Submit the correct lowercase letter, the actual routine's final state, and model A's final state, separated by underscores.

Open these files: ghost-circuit.img.bin, MEMORY-MAP.txt, 07-models.txt, RE-TOOLKIT.txt.

1. Read 07-models.txt. Trace AABC from state zero through the actual instructions and through model A, keeping separate columns. Do not assume a friendlier restart rule than the code implements.
2. Also test X from prior state 3 against model C. Use that boundary case to confirm the terminal-state behavior. For invalid prior states, check the unsigned JA path.
3. Finish your RE-TOOLKIT worksheet or export Notebook: function map, record layout, indirect edge, byte transform, call trace, state table and one counterexample. You have built a reusable analysis method, not just collected flags.

Read what happened

Use AABC as a distinguishing input between the actual state machine and the proposed restart-on-A model. Keep two state columns and process each event once. The second A is where their reset rules diverge. Then test a nonmatching event from state 3 to distinguish the actual terminal behavior from the candidate that forgets completion.

The correct model is the one that survives these counterexamples and matches the unsigned invalid-state handling. Preserve both the failing model and the smallest input that disproves it. This is a reusable RE technique: one carefully chosen boundary sequence can be more informative than many ordinary inputs. Your final toolkit should contain the map, field layout, indirect edge, call-value ledger and state table, not only the formatted response.

Hints, one at a time

1. Models A and B differ when a mismatch byte is A. The real routine has no instruction that reprocesses that byte as a fresh start.
2. On AABC, the second A arrives in state 1. The real routine returns zero; model A returns one. Trace B and C separately after that.
3. Model C loses the completed state on another input. The actual routine returns three immediately whenever the prior state is three. Your flag uses the AABC final states, not this X test.

Response format: `SPECTRA{letter_actual_modela}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Keep the evidence: write the exact source offset or instruction, the operation you applied, and a sentence explaining why it answers this stage. Keep intermediate values labeled so the next stage does not confuse an input with a return value.

Close the investigation

Export a note or selected evidence bundle containing artifact identities, your coordinate/value ledgers and the smallest observations needed to reproduce your conclusions. Describe unfinished evidence explicitly. Before moving on, explain one stage aloud without looking at a hint: which bytes mattered, what rule connected them, and what the result does not prove.

19. Hollow Signal: a decoder, a parser and a boundary proof

An encoded capture is only the start. Recover the byte recurrence, parse length-delimited records, audit two boundary guards and explain a revised capture. Keep capture offsets, decoded offsets and code VAs in separate columns. A readable string is not the end of the investigation.

Open the public campaign casebook. Exact public practice files and hashes are in `practice/ctf/005/`. Save your current investigation before changing the main file.

Stage	Skill to carry away
01 · Bound the evidence	Headers + coordinate systems
02 · Recover the moving key	Loop state + rolling XOR
03 · Recover a typed value	Decoded bytes + TLV field widths
04 · Walk past the unknown	Length-delimited parsing + unknown records
05 · Prove the boundary	Remaining length + validation order
06 · Find the counterexample	Unsigned wraparound + reachability
07 · Explain the mutation	Raw diff + semantic comparison
08 · Write what you can prove	Evidence + limitations + reusable tooling

Use the campaign’s Local trace worksheet when a stage calls for the rolling-key calculation. It runs on your chosen small capture in the browser. Ordinary repeating XOR in Decoder lab is a different algorithm.

Stage 01 · Bound the evidence

The question. An encoded configuration sits inside a larger capture. Find its absolute file offset and byte count from the header. Submit each as four hex digits. A readable string elsewhere in the file is not automatically configuration data.

Open these files: capture.bin, MEMORY-MAP.txt, FORMAT-NOTES.txt.

1. Download MEMORY-MAP.txt and FORMAT-NOTES.txt. Open capture.bin in SPECTRA Hex explorer. Inspect header fields at file offsets 0x06 and 0x08.

2. Read the count as a two-byte little-endian value, and the data offset as a four-byte little-endian value. Calculate the exclusive end: offset + count.
3. Record the bounded range and last included byte in Notebook. Open the Casebook link for the full lesson and coordinate-system explanation.

Read what happened

Treat FORMAT-NOTES as a supplied analyst contract. You are not expected to infer this whole format from nothing. In Hex explorer, use the file-offset column; do not decode the header as instructions.

A count is a quantity, not an address. A half-open interval [start,end) includes start and excludes end. For a different example, start 0x10 and count 3 include 0x10, 0x11, 0x12; end is 0x13.

There are three coordinate systems in this mission: capture file offsets; decoded buffer offsets starting at zero; and code virtual addresses starting at 0x7000. A decoded byte i corresponds to capture offset data_offset+i, because this transform preserves length. That does not make it a code VA.

Write a small header ledger: field, width, stored bytes, interpreted value, and source of meaning. Check start <= file_size and count <= file_size-start before taking a slice. Do not include padding before the data or the readable tail after it.

Hints, one at a time

1. FORMAT-NOTES identifies field widths. The u16 at +6 is COUNT; the u32 at +8 is OFFSET. Both are little endian.
2. For bytes aa bb, a little-endian u16 is aa + 256*bb. Do not reverse the characters within each byte.
3. The range ends just before 0x3f. The flag uses its START and COUNT, not its last offset or exclusive end.

Response format: `SPECTRA{range_oooo_nnnn}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Check your reasoning: Can you explain why the last included offset is one less than the exclusive end, and why the flag asks for a count instead of that end?

If your result differs: Seeing Raw binary / Unknown is expected. Use Hex explorer, not automatic PE structure analysis. Put 0x before hexadecimal input when a SPECTRA field accepts either decimal or hex.

Beyond this case: This habit transfers to overlay extraction, resource recovery, packet parsing and memory carving: bound the evidence before interpreting it.

Stage 02 · Recover the moving key

The question. The decoder changes its key after every byte. Trace the first iteration and find the key used for the SECOND byte. Submit two hex digits. This builds on CTF 004's rotations, now with state carried around a loop.

Open these files: hollow-signal.img.bin, capture.bin, MEMORY-MAP.txt.

1. Open hollow-signal.img.bin. Decode the decoder row in MEMORY-MAP: offset 0x0, length 0x21 (33 decimal), base VA 0x7000, x86-64, Little endian; select All decoded instructions.
2. Use the register contract in the map. Follow source load, XOR, destination store, key rotation, key addition and index update in instruction order. Get the initial key from capture header +5.
3. Trace one eight-bit key update on paper. The key BEFORE the update decodes byte zero; the updated key decodes byte one. Save both values for the next stage.

Read what happened

Read the map before the assembly. RCX and RDX are pointers, R8D is a count, and R9B holds an eight-bit key. EAX starts at zero and indexes both source and destination. R10B is temporary storage for one byte.

Separate the data transform from the key update. The data path loads source[index], XORs with the current key, and stores one output byte. Only then does the key path rotate and add. Reversing that order shifts the entire key stream.

ROL is a rotation, not a shift: the high bit wraps around. Because the operand is R9B, the operation is eight bits wide. ADD to that same register discards carry beyond eight bits. A 32-bit rotation is a different function even if the seed fits in a byte.

The INC/CMP/JB sequence controls the loop count. With a zero count the initial TEST/JE (JE is also called JZ) skips all reads. With a positive count, indices are 0 through count-1. The routine trusts its caller to provide real buffers of that size; this is not validation of the capture header.

Hints, one at a time

1. The initial seed is a7. R9B is eight bits. XOR uses the current key before ROL and ADD change it.
2. ROL8 by one moves bit 7 into bit 0: ((key << 1) | (key >> 7)) & 0xff. Then add the immediate and keep the low eight bits.
3. Rotating a7 left by one gives 4f. Add the immediate shown in ADD R9B, then submit that new key, not the first decoded byte.

Response format: `SPECTRA{key_xx}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Check your reasoning: Label key_before and key_after in your trace. Which one is used by the XOR on the first iteration? What prevents a read for count zero?

If your result differs: A blank function table does not mean no code. Decode the exact raw row and select All decoded instructions. R9B is not a typo for R9D; that suffix changes the arithmetic width.

Beyond this case: Stateful decode loops appear in obfuscated strings and configuration loaders. A register ledger lets you reconstruct them without running the original program.

Stage 03 · Recover a typed value

The question. Decode the original bounded configuration, then recover the first interval value. Submit its u16 value as four hex digits. Use the local trace worksheet after tracing the first two bytes yourself; no programming is required.

Open these files: capture.bin, FORMAT-NOTES.txt.

1. Open the Casebook's Local trace worksheet. Choose capture.bin and enter the offset, count and seed from its header, plus the rotate count and add constant from the decoder. Trace, then save decoded.bin.
2. Open decoded.bin in SPECTRA Hex explorer. FORMAT-NOTES defines each record as type, VALUE length, then value bytes. Find type 0x10 and read its two value bytes as little endian.
3. Record the type, length, value and next record offset. The format notes supply the meaning: seconds. Do not include the two header bytes in the interval value.

Read what happened

Use the helper to check your model, not to choose it. It implements: output byte = input byte XOR current key; then rotate the key left in eight bits, add your constant, keep eight bits. Enter the actual constants from your disassembly. It displays each input, key, result and next key.

Before trusting the full output, compare its first two rows with your own trace. A plausible first byte alone is weak evidence. The known record lengths and the rest of the structure provide independent checks.

Save the output and use the MAIN Open file button to inspect decoded.bin in SPECTRA. Its first byte is offset zero. That byte originally came from the capture's data offset, but the exported file has no capture header.

TLV means Type-Length-Value. Here both type and length are one byte. The length counts the value only. For the first record, mark the two-byte header separately from the two-byte number; then apply the little-endian rule from CTF 001. Write decimal seconds in your notes and hexadecimal in the requested flag.

Hints, one at a time

1. The helper is linked above the stages in the web casebook. SPECTRA's ordinary XOR decoder does not implement this rolling-key recurrence.

2. The first four decoded bytes are 10 02 2c 01. The first TWO are tag and length; only the last two hold the interval.
3. A little-endian WORD formed from 2c 01 is 0x0100 + 0x002c. Pad the resulting hex value to four digits.

Response format: `SPECTRA{interval_hhhh}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Check your reasoning: Can you explain where the unit 'seconds' came from? It is supplied by the format contract, not something an arbitrary two-byte value proves on its own.

If your result differs: If the output starts with nonsense, check the original capture (not revised), byte range, eight-bit rotate, seed and update order. Keep the helper's numbers explicitly 0x-prefixed. Use the hints as a checkpoint rather than restarting the whole mission.

Beyond this case: This is how configuration extractors become reproducible: a bounded transform, explicit types and validation against more than one known field.

Stage 04 · Walk past the unknown

The question. Walk every TLV record in the original decoded buffer. Find the final cursor and the LAST record's unknown type. Submit the cursor as four hex digits and the type as two. Unknown semantics do not prevent you from locating the next record.

Open these files: FORMAT-NOTES.txt, ANALYST-WORKSHEET.txt.

1. Keep the original decoded.bin in Hex explorer. Start cursor at 0; read a one-byte tag and one-byte VALUE length. Write each row in ANALYST-WORKSHEET.txt or Notebook.
2. For each record, validate and advance by 2 + length. Do not align, search for a NUL, or stop on an unknown type. The endpoint label has no terminator.
3. Stop when cursor equals the decoded size. Submit that final cursor and the final record's unknown tag. Keep its raw bytes and label its meaning unknown.

Read what happened

Draw columns: record start, tag, value length, value start, next cursor and interpretation. At every row, value_start = cursor+2. The next cursor is value_start+length. Each row should start exactly where the previous row ended.

Count bytes, not displayed characters. A field may contain NUL, non-ASCII values or arbitrary binary data. The hex pane is the authoritative representation; a text preview is just one interpretation.

For the supplied format, unknown tags use the same length convention. You may skip them only after the declared value fits. 'Unknown' describes semantics, not whether its bytes exist. Preserve the raw tag and value in your notes.

There is no end sentinel. Reaching exactly buffer_size is success. Cursor beyond buffer_size is a malformed record, and a single byte left is a truncated header. The final cursor is not an offset you should read.

Hints, one at a time

1. Record starts are decoded offsets 0, 4, 7 and 0x1a. These are NOT capture offsets or code VAs.
2. The third record's length is 0x11 (17 decimal). Its value is exactly that many bytes; the following byte begins a new tag.
3. The last value is three bytes and includes 00. Skip by length anyway. Final cursor is an exclusive end, so it equals the 31-byte output size.

Response format: `SPECTRA{end_oooo_tt}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Check your reasoning: Why is stopping at the first 00 byte wrong here? Why can you compute a record boundary without knowing what that record means?

If your result differs: If you reach the readable TAIL text, you decoded beyond the range from stage 01. If your third record seems to overlap the last, check that length excludes its two header bytes.

Beyond this case: The same ledger works for tagged configuration blobs, binary protocols, resource chunks and extensible file formats. Preserve uncertainty without losing your place.

Stage 05 · Prove the boundary

The question. Analyze guard_after, a pure length-check predicate. With total=31 and cursor=26, what is the largest VALUE length it accepts? Submit two hex digits. Account for the two-byte record header before reasoning about the payload.

Open these files: hollow-signal.img.bin, MEMORY-MAP.txt, BOUNDARY-CASES.txt.

1. Open hollow-signal.img.bin. Decode guard_after: offset 0xc0, length 0x1b (27 decimal), base VA 0x70c0, x86-64, Little endian. Select All decoded instructions.
2. Use the map's contract: ECX=total, EDX=cursor, R8D=value length. Trace each compare before its subtraction. JA means unsigned above; JB means unsigned below.
3. Work through exact-fit and one-too-many in BOUNDARY-CASES.txt. Record the largest valid value length and explain why the next larger value fails.

Read what happened

Rewrite the routine in plain language one branch at a time: reject a cursor beyond total; calculate remaining bytes; reject fewer than two header bytes; subtract the header; reject a value length larger than the remainder. The initial zero return becomes one only on the accepted path.

Do not treat SUB as automatically safer than ADD. If cursor were larger than total, unsigned total-cursor would wrap to a huge value. The preceding comparison is what makes this subtraction valid. The same reasoning applies before subtracting the header.

Distinguish the header-fit question from the value-fit question. With zero total bytes, even a zero-length VALUE does not fit because its header still needs space. With two total bytes, a zero-length value at cursor zero does fit.

JA and JB use unsigned ordering. They do not mean signed greater/less. At the final CMP R8D,ECX, equality is allowed because the code branches only on above. Record accepted and rejected examples so your explanation covers the boundary, not just the middle.

Hints, one at a time

1. First establish cursor <= total. Only then is total-cursor a valid remaining-byte count.
2. At cursor 26 of total 31, five bytes remain. The tag and length header consumes two of them.
3. The final comparison accepts equality and rejects unsigned values ABOVE the remaining payload space. Test the exact fit and then add one.

Response format: `SPECTRA{limit_xx}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Check your reasoning: Can a zero-length value be valid? Under which total/cursor conditions? Why must cursor be checked before subtraction?

If your result differs: Keep all vector numbers in the units given: decimal 31/26 or hex 0x1f/0x1a. The question asks for maximum VALUE bytes, not total record bytes.

Beyond this case: These checks help you review configuration parsers and distinguish actual out-of-bounds risks from code that only looks suspicious. MITRE's CWE-125 describes the class of reads beyond an intended buffer; this predicate itself performs no read.

Stage 06 · Find the counterexample

The question. Compare guard_before with guard_after using the wide-length vector. Submit the old and fixed return values, separated by an underscore. This is a paper trace of two predicates, not a request to trigger a crash or execute a file.

Open these files: hollow-signal.img.bin, MEMORY-MAP.txt, BOUNDARY-CASES.txt.

1. Decode guard_before: offset 0x80, length 0x0f (15 decimal), base VA 0x7080. Inputs are total=0x20, cursor=0x1e and value length=0xffffffff.
2. Trace both ADD instructions in 32 bits, then the unsigned comparison and SETBE. Separately trace guard_after with the same inputs. Record each return.
3. Explain why the old check is wrong, then state the scope limit: this u32 API vector is not representable by the capture's u8 length field.

Read what happened

Keep the arithmetic width visible in your notes. EDX and R8D are 32-bit registers. When an addition exceeds 0xffffffff, the stored value wraps modulo 2^32. The processor does not preserve an unlimited mathematical integer for the later CMP.

The old code checks a sum AFTER it has already wrapped. A very large length can therefore look like a small end position. SETBE only knows the final unsigned comparison result; it cannot recover the intended mathematical sum.

The fixed code compares length against an already-validated remainder. For this vector, only the header fits, so no value bytes remain. The large length is rejected without needing to add it to the cursor.

Now ask whether this counterexample can reach the function in a real call chain. In our capture, the TLV length is one byte, so 0xffffffff cannot come from that field. The generic u32 helper is a separate exercise. Your report must not turn a wider API bug into an unsupported claim about the supplied capture, SPECTRA, or real malware.

Hints, one at a time

1. 32-bit ADD retains only the low 32 bits. SETBE writes 1 if the preceding unsigned comparison is below OR equal, otherwise 0.
2. For the old guard, calculate (0x1e + 2 + 0xffffffff) modulo 0x100000000. Compare that small wrapped result with total 0x20.
3. The old guard sees 0x1f <= 0x20. The fixed guard sees no value bytes left after the header and rejects the huge declared length. Submit the two boolean return values.

Response format: `SPECTRA{guard_old_fixed}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Check your reasoning: What did the counterexample prove locally? What additional caller/input evidence would you need before claiming an exploitable parser issue?

If your result differs: Do not use a calculator's unbounded result without masking to 32 bits. Do not interpret 0xffffffff as signed -1: this contract and comparison are unsigned.

Beyond this case: Counterexamples, operand widths and reachability analysis belong together in vulnerability research. A credible report explains both the failure and the conditions required to reach it.

Stage 07 · Explain the mutation

The question. The revised capture changes three bytes. Separate metadata from the decoded configuration change and recover the NEW interval. Submit four hex digits. A byte diff locates changes; decoding explains what they mean.

Open these files: capture.bin, capture-revised.bin, FORMAT-NOTES.txt.

1. Open capture.bin as the MAIN file. In Binary diff, Open second binary > capture-revised.bin > Compare. Use Raw byte comparison; no function matching is needed for these data files.
2. Record each absolute changed offset and before/after byte. Use FORMAT-NOTES to classify header metadata separately from encoded data. Convert encoded offsets to decoded offsets by subtracting the data start.
3. Decode capture-revised.bin in the Local trace worksheet using its header and the same code-derived transform. Save it separately, open in Hex explorer and read the interval field. Compare your notes with the original.

Read what happened

Use the byte table even if Matched functions is empty. Both captures are data, not executable images, so function recovery adds nothing to this comparison. The raw table compares bytes at the same absolute offsets; it is not an insertion-alignment algorithm.

Map each change back to the format. A revision byte may help label versions but does not itself tell you what behavior changed. Data-offset and count fields should also be checked before assuming both payloads occupy the same range.

The rolling key depends on prior KEY state, not prior plaintext or ciphertext. Therefore a changed data byte does not change all subsequent keys. With the same seed, offset, count and recurrence, compare the recovered fields directly.

Keep original and revised decoded files distinguishable. A hash, filename and small record ledger prevent you from reading the old file while believing it is the new one. Describe the numeric interval change only under the supplied seconds contract; the files do not implement a timer or prove runtime network behavior.

Hints, one at a time

1. The three changed capture offsets are 0x04, 0x22 and 0x23. The first is the revision counter, outside the encoded range.
2. The encoded data still starts at 0x20. Changes at 0x22 and 0x23 therefore affect decoded offsets 2 and 3, the first record's value.
3. The revised interval bytes decode to 58 02. Read them as one little-endian u16, not two unrelated numbers or the old interval.

Response format: `SPECTRA{interval_hhhh}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Check your reasoning: Why did only two encoded payload bytes change? Would that still be guaranteed for a decoder whose next key depends on the previous plaintext byte?

If your result differs: Use MAIN Open file for capture.bin, then Open second binary for the revised copy. If SPECTRA shows decoded.bin as the main file, you are comparing different coordinate systems. No raw code decode is required here.

Beyond this case: This method helps compare malware configurations, protocol captures and firmware resources while separating metadata churn from meaningful changes.

Stage 08 · Write what you can prove

The question. Review the three candidate reports and choose the one supported by the artifacts. Submit its lowercase letter. Finish your analyst worksheet: the checked letter earns progress, but the explanation is the skill you take into future work.

Open these files: REPORT-CANDIDATES.txt, ANALYST-WORKSHEET.txt.

1. Read REPORT-CANDIDATES.txt. Check each claim against your header ledger, key trace, record map, boundary tests and mutation notes.
2. Separate observed bytes, supplied format meanings, derived results and unproven runtime behavior. A string is not a connection; an unknown tag is not a credential by default.
3. Choose the defensible report, export your SPECTRA Notebook, then use Collect patch 005 after all eight stages are solved. The flag checker does not grade your written report.

Read what happened

For each statement, attach its evidence source. Header bounds come from capture bytes interpreted under the format notes. Key recurrence comes from instructions. Numeric values come from the decoded record widths. The code and data alone do not establish a live campaign, actor or network session.

Use limitations as precision, not as an apology. 'Unknown type f0, raw value retained' is a stronger statement than inventing a meaning. 'This vector demonstrates wraparound in the generic predicate; reachability through the u8 field is absent' is stronger than an unsupported exploit claim.

A useful final note includes sample hashes, exact ranges, a short pseudocode model, a record ledger, a boundary counterexample, the changed field, and the next question you would investigate. Another analyst should be able to reproduce your conclusion without needing your flag.

You have combined earlier CTF skills into a small configuration-analysis workflow. Keep your worksheet as a template. In future missions you can replace supplied schema notes with inferred field meanings and compare several captures, while keeping the same evidence discipline.

Hints, one at a time

1. Reject claims of network activity: none of the supplied teaching code makes a network call. The endpoint is only recovered data.
2. Reject models that treat any unknown tag, NUL or ff byte as automatically invalid. Length and the supplied schema control parsing.

3. The strongest report preserves the unknown type's raw bytes and distinguishes the generic u32 guard flaw from the capture's one-byte length field.

Response format: `SPECTRA{report_letter}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Check your reasoning: Can you distinguish supplied knowledge from inferred behavior in every paragraph of your report? Can another person reproduce your decode?

If your result differs: There is one supported candidate. The flag uses only its lowercase letter inside the stated wrapper. Do not paste the candidate text, explanations or punctuation after the closing brace.

Beyond this case: These notes can become the basis of a configuration extractor, a defensive signature review or an incident-response handoff. Clear scope protects downstream analysts from acting on claims the evidence does not support.

Close the investigation

Export a note or selected evidence bundle containing artifact identities, your coordinate/value ledgers and the smallest observations needed to reproduce your conclusions. Describe unfinished evidence explicitly. Before moving on, explain one stage aloud without looking at a hint: which bytes mattered, what rule connected them, and what the result does not prove.

20. Iron Veil: collisions, indirect calls and missing coverage

This case joins a short name hash to a resolver and gated dispatcher. You will keep colliding candidates, apply a second discriminator, derive path constraints, interpret a callback contract and compare a boundary-changing revision. The supplied observation set is deliberately incomplete; explaining its gap is part of the work.

Open the public campaign casebook. Exact public practice files and hashes are in `practice/ctf/006/`. Save your current investigation before changing the main file.

Stage	Skill to carry away
01 · Recover the name hash	Rotation + word width + terminators
02 · Keep both candidates	Table parsing + little endian + collisions
03 · Follow the second test	Resolver control flow + compound conditions
04 · Write the path constraints	Short-circuit gates + unsigned comparisons
05 · Read the return contract	Indirect call + argument tracking + status semantics
06 · Test the exact boundary	Raw byte diff + branch semantics + controlled inputs
07 · Find the coverage gap	Trace evidence + reachability + test coverage
08 · Write what is supported	Evidence synthesis + limitations + analyst handoff

The campaign’s Name hash worksheet supplies a local arithmetic check after you derive the algorithm. It does not identify arbitrary malware APIs or prove a hash is globally unique.

Stage 01 · Recover the name hash

The question. A short routine converts a name to a 16-bit value. Recover its rule, then calculate the hash of the first NUL-terminated name in label.bin. Submit four lowercase hex digits after hash_.

Open these files: iron-veil.img.bin, label.bin, MEMORY-MAP.txt, FORMAT-NOTES.txt.

1. Read MEMORY-MAP.txt and FORMAT-NOTES.txt. Open iron-veil.img.bin in SPECTRA Disassembly. Decode file offset 0x000, byte length 0x16 (22 decimal), base VA 0x9000, x86-64, Little endian.

2. Trace EAX's initialization, the byte load, the terminator check, and the AX rotation/addition. In Hex explorer open label.bin and identify where the name ends.
3. Work through its first two bytes on paper. Open the Casebook's Name hash worksheet to finish using the parameters you recovered. Keep the hash to four hex digits.

Read what happened

Start with the supplied calling contract: RCX points to a readable NUL-terminated string, and EAX holds the result. This is an isolated teaching routine, not a whole loader. Select All decoded instructions if SPECTRA's function filter hides the loop.

XOR EAX,EAX initializes the accumulator. MOVZX EDX,byte [RCX] reads one byte without a sign extension. TEST DL,DL and the conditional jump decide whether to stop. Because that check happens before the update, neither the NUL nor any trailing bytes contribute.

ROR AX,3 rotates a 16-bit word: bits leaving the right edge return at the left edge. ADD AX,DX keeps only 16 result bits. After incrementing RCX, the next iteration uses the updated hash. For a separate worked example, AB hashes through 0041, then ROR16(0041,3)=2008, then 2008+0042=204a.

Open label.bin as data and copy the first name with its exact case. The text after the NUL is deliberately outside the input. Trace two bytes in your Notebook before using the local worksheet. Enter the recovered seed and rotation there; the tool shows every intermediate value so you can compare your model.

The arithmetic here is a small name hash, not encryption and not a security-grade digest. There is no secret key to recover. Many different names can produce the same 16-bit output; the next stage makes that limitation visible.

Hints, one at a time

1. EAX starts at zero. AX is its low 16 bits. MOVZX makes the loaded byte an unsigned value; ROR wraps bits within AX, not a 32-bit register.
2. Each nonzero byte updates h = (ROR16(h,3) + byte) modulo 65536. The zero byte stops the loop BEFORE a rotate or addition. Preserve capitalization.
3. For a different name, AB: A produces 0041; rotating that right by three produces 2008; adding B (42 hex) produces 204a. Apply that process only to the name before the first NUL in label.bin.

Response format: `SPECTRA{hash_hhhh}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Check your reasoning: Can you identify the instruction that excludes the terminator and explain why using ROR32 would produce a different answer?

If your result differs: Raw binary / Unknown is expected. Decode only the code range, not label.bin. Byte length is 0x16, not 0x22. The name worksheet works entirely in this browser and accepts at most 64 printable ASCII characters.

Beyond this case: This procedure transfers to defensive analysis of hashed imports and custom lookup schemes: reconstruct the recurrence, verify it with a small trace, then test a bounded candidate list.

Stage 02 · Keep both candidates

The question. request.bin supplies a target hash. In the five-row symbol table, find EVERY row whose stored hash matches it. Submit the two row IDs in ascending order, each with two hex digits. Do not use the request's name length yet.

Open these files: request.bin, iron-veil.img.bin, FORMAT-NOTES.txt.

1. In Hex explorer read the u16 request hash at request.bin offset 0x04. Use FORMAT-NOTES.txt to interpret it as little endian.
2. Open iron-veil.img.bin. The table starts at file offset 0x300, with five rows of 0x0c bytes. Read each row's ID at +0 and hash at +2.
3. Follow each matching row's name FILE offset at +8, record its name and length, and recalculate the hash with the worksheet. Keep both candidates; a collision is expected.

Read what happened

Use the format note as supplied knowledge, not something you are expected to guess. A row contains an ID, a length, a stored hash, a target VA and a name file offset. Write those labels above the bytes before interpreting values.

For row index i starting at zero, its file offset is 0x300 + i*0x0c. The hash begins two bytes later. For an unrelated example, bytes 34 12 store u16 0x1234. The order of characters inside each printed byte does not change.

Read the request's hash independently, then compare every row. Only after collecting matches should you follow the name offsets. Those offsets refer to the same image file; they are not VAs. Names end at NUL and their stored lengths exclude that NUL.

Recalculate each candidate name with the recovered hash rule. Two different names really do produce the same value here; this is not a corrupted artifact. The supplied table deliberately demonstrates why a hash alone cannot prove identity.

Do not discard a candidate merely because its name sounds less convincing. At this stage the evidence is the hash comparison alone. Keep both rows in your lookup ledger; the next stage asks what the resolver itself does with them.

Hints, one at a time

1. The table occupies [0x300,0x33c). Row starts are 0x300, 0x30c, 0x318, 0x324 and 0x330. Do not mistake the target VA field for the hash.
2. A stored u16 with bytes cc a3 represents 0xa3cc. Compare whole byte values, not individual reversed characters.
3. Two adjacent rows match the requested hash. One name has 12 characters, the other 9. Record their ID bytes, not their positions expressed as file offsets.

Response format: `SPECTRA{rows_aa_bb}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Check your reasoning: Do both stored values match a calculation over their actual names? Have you kept the candidate set separate from the final selected target?

If your result differs: Use the image's absolute FILE offsets. The name pointer field at +8 is a file offset; the target at +4 is a VA. You are reading data, so an empty automatic disassembly result does not block this stage.

Beyond this case: In a real investigation, compare a hashed value against a relevant, versioned export catalog and retain ambiguity until code or runtime evidence resolves it.

Stage 03 · Follow the second test

The question. The resolver tests more than a hash. Trace its comparisons using request.bin and the supplied table, then submit the selected target VIRTUAL ADDRESS as eight hex digits. This stage asks for a VA, not a file offset.

Open these files: iron-veil.img.bin, request.bin, MEMORY-MAP.txt, FORMAT-NOTES.txt.

1. Decode iron-veil.img.bin at file offset 0x080, length 0x20 (32 decimal), base VA 0x9080, x86-64, Little endian.
2. Use the calling contract: RCX=table, EDX=5 rows, R8W=request hash, R9B=request name length. Follow both comparisons for each matching candidate.
3. Read the selected row's target at +4. Convert its VA to a file offset only to inspect the target; submit the VA itself and explain why the other row was skipped.

Read what happened

Separate caller inputs from code behavior. The request supplies the desired hash and length. The table stores precomputed metadata. This routine compares that metadata; it does not hash the strings again or perform a full string comparison.

Trace the first matching hash candidate through the length comparison. If it fails, follow the next-row path: RCX advances by 12 and the remaining count decreases. Continue until both conditions match or the row count is exhausted.

The successful path loads the u32 target field into EAX. A 32-bit register write clears the upper half of RAX, so this small synthetic VA is returned without high-bit residue. A failed lookup returns zero, and the harness only dispatches after a valid nonzero result.

The extra length comparison separates these two supplied candidates. It does not make hash-plus-length mathematically collision-free, and the implementation returns the first matching row if duplicates exist. Record that limitation rather than claiming exact-name verification.

Use the memory map to inspect the selected stub. Keep both coordinates in your notes: target VA for control flow, file offset for locating bytes on disk. The flag specifically requests the VA.

Hints, one at a time

1. The resolver compares WORD [RCX+2] with R8W, then BYTE [RCX+1] with R9B. Both tests must pass.
2. The requested length is the byte at request.bin +6. One hash candidate has length 12, the other 9. The successful return loads DWORD [RCX+4].
3. The selected target lies in the stub region. In this image VA = 0x9000 + file offset. Pad the target VA to eight digits; do not submit its file offset.

Response format: `SPECTRA{va_vvvvvvvv}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Check your reasoning: Can you explain the successful and exhausted-table returns, and why the chosen target is unique only within this supplied catalog?

If your result differs: If you get a plausible value near 0x500, you probably converted to a file offset. The requested value is its mapped VA. Decode range replaces the previous routine; use the map to return to earlier code.

Beyond this case: This is the same evidence discipline needed when following indirect calls through custom dispatch tables, hashed imports or plug-in registries.

Stage 04 · Write the path constraints

The question. For the ORIGINAL dispatcher, trace configs A, B and C in that order. Which return code does each produce? Submit three two-digit codes. Also record whether the indirect callback is reached, even though that note is not part of the flag.

Open these files: config-A.bin, config-B.bin, config-C.bin, PATH-CASES.txt.

1. Read the configs as 16-byte records using FORMAT-NOTES.txt. Decode the original image at offset 0x100, length 0x44 (68 decimal), base VA 0x9100, x86-64, Little endian.
2. Track the enabled bit, offline bit and unsigned elapsed-versus-interval comparison in code order. Stop tracing each case when it reaches RET.
3. Write the exact conditions required to reach CALL RDX. Use the provided dispatcher return legend, then submit results in A/B/C order.

Read what happened

Build one row per config with columns for flags, elapsed, interval and status. Reserved bytes are not extra fields. Multibyte fields are little endian; ticks are abstract units, so do not silently describe them as seconds.

Follow the dispatcher from its entry. It allocates stack space for an eventual call, but allocation itself is not callback activity. The first flag test can choose an early return; the next test can choose another. Later fields may exist in the input without ever being used on that path.

At the time gate, CMP sets flags as if subtracting the interval from elapsed. JB uses the unsigned below condition. Write the true and false conditions in words, including equality, rather than relying on a vague label like 'time check'.

For each case, stop at RET and record the dispatcher code from the supplied legend. Do not continue into the next displayed block simply because it appears below in the linear listing. Branch targets and returns determine execution order.

Finally combine the three gates into one predicate for reaching CALL RDX. Distinguish that reachability predicate from successful completion: the callback can still report failure after being reached. Configs A/B/C intentionally exercise earlier exits.

Hints, one at a time

1. Keep bit tests separate: bit 0 means enabled and bit 1 means offline. A nonzero flags byte does not mean every condition is true.
2. A branch taken at an earlier gate can return before a later comparison is evaluated. Read CMP's operand order and use unsigned JB semantics.
3. A fails the enabled gate; B passes enabled but takes the offline path; C passes both flag gates but has elapsed below its interval. Look up each path's return code in FORMAT-NOTES.

Response format: `SPECTRA{paths_aa_bb_cc}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Check your reasoning: Does each path ledger state which later gates were not evaluated? Can you explain equality without changing from unsigned to signed arithmetic?

If your result differs: Open the ORIGINAL image as the main file. A config is data, not the code image. Your flag contains dispatcher return codes, not flags bytes, elapsed values or callback counts.

Beyond this case: Path ledgers help explain dormant features, feature flags, environment checks and delayed behavior without claiming that every discovered routine ran.

Stage 05 · Read the return contract

The question. Trace the ORIGINAL dispatcher with config D and the selected callback. Submit the dispatcher result followed by the number of callback entries, both two hex digits. The callback's raw status is not the requested dispatcher result.

Open these files: config-D.bin, iron-veil.img.bin, FORMAT-NOTES.txt, MEMORY-MAP.txt.

1. Read config D. Trace dispatch using its existing range and callback VA from stage 03. Track the value copied into ECX immediately before CALL RDX.
2. Decode the selected stub using MEMORY-MAP.txt. Follow its returned EAX back to the dispatcher's TEST and conditional branch.
3. Apply the provided callback contract: exactly zero is success. Record callback EAX, dispatcher EAX and call count separately; submit the last two.

Read what happened

Treat CALL RDX as a transfer to the target established by the resolver contract. The selected stub is not an OS API. The dispatcher reserves 0x28 stack bytes for shadow space and alignment before the call and restores its stack before returning.

Track the argument transformation: RCX initially points to the config, but MOV ECX,[RCX+12] replaces it with a 32-bit status value. The stub receives that status in its first argument register. The instruction reads through the old pointer before writing ECX.

Decode the stub separately. Its MOV and RET copy the argument to the return register without performing external work. Follow the return back to the instruction after CALL, then evaluate TEST/JNE using the callback contract supplied in FORMAT-NOTES.

Keep three different facts in the ledger: the callback's raw 32-bit EAX, the dispatcher's small result code, and how many times the callback was entered. They answer different questions. Signed interpretation can describe the same bits, but this branch only distinguishes zero from nonzero.

The name QueueNote is a label, not evidence of networking or a real queue implementation. Inspect the body before attributing behavior. This deliberately tiny stub lets you practice call reasoning without running native code or contacting a service.

Hints, one at a time

1. Windows x64 integer arguments begin in RCX, RDX, R8 and R9. Here dispatch replaces ECX with the config's status before the call; the stub receives that value.
2. The stub is MOV EAX,ECX; RET. Back in dispatch, TEST EAX,EAX checks zero versus nonzero. A value with the high bit set is still nonzero.
3. Config D reaches the callback once. Its returned status takes the failure path. The flag asks for dispatch's failure code from the legend, then a call count of one.

Response format: `SPECTRA{result_rr_cc}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Check your reasoning: Can you explain why a call can be reached yet yield a failure result, and identify the source of the status contract instead of inferring it from the ABI?

If your result differs: Do not submit the eight-digit raw callback status. Use the two-digit dispatcher code plus a two-digit count. For code setup, use the exact separate dispatch and stub ranges in MEMORY-MAP.

Beyond this case: This skill helps when tracing error handling, wrappers and callback chains in unfamiliar binaries: recover the callee's contract and then inspect how the caller interprets it.

Stage 06 · Test the exact boundary

The question. One branch opcode differs between the ORIGINAL and REVISED images. With the SAME config E, find each dispatcher's return code. Submit original then revised as two-digit codes. Explain the changed behavior at equality in your notes.

Open these files: iron-veil.img.bin, iron-veil-revised.img.bin, config-E.bin, PATH-CASES.txt.

1. Open the original image as the main file and the revised image as the second Binary diff file. Inspect the raw changed-byte table; a blank function-matching table does not block raw comparison.
2. Disassemble dispatch in each version separately using offset 0x100, length 0x44 (68 decimal), base VA 0x9100. Compare the changed jump's condition and destination.
3. Trace config E through both versions with the same callback VA. Test one tick below, equal to, and above the interval on paper. Submit the two equality-case results.

Read what happened

Use Binary diff first to locate the changed byte. Keep its absolute file offset separate from the mapped VA shown in disassembly. A raw comparison does not require successful function matching or a PE header.

For instruction context, make each image the main Open file in turn and repeat ALL raw-range settings. Loading the second comparison file alone does not make it the main disassembly source. Record each mnemonic and destination before switching files.

Read the unsigned branch condition precisely. A below test excludes equality; a below-or-equal test includes it. The surrounding CMP and target block remain the same, so the change adds one boundary case to the early return.

Use config E as the controlled pair: same flags, elapsed, interval, status and callback target. Trace both versions. Then check one tick below and above: those inputs help verify that your explanation is specifically about equality, rather than a claim that all calls disappeared.

A changed opcode does not establish motive. Your defensible conclusion describes the condition and its observable consequence in this supplied routine. Whether a real patch is a fix, regression or intentional policy change requires more context.

Hints, one at a time

1. There is one changed byte. Interpret it as part of the conditional jump immediately after the elapsed/interval comparison, not as a standalone config value.
2. JB means unsigned below; JBE means unsigned below OR equal. The jump destination is unchanged. Equality is the input that distinguishes them.
3. Config E has elapsed equal to its interval and a zero callback status. One version reaches the success path; the other returns not due before the call. Use the dispatcher legend.

Response format: `SPECTRA{boundary_aa_bb}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Check your reasoning: Have you changed only the image version in the distinguishing test? Can you predict the below/equal/above outcomes without running the file?

If your result differs: The flag uses RETURN CODES, not the diff offset or opcode bytes. Keep original first, revised second. To see raw changes, scroll to the byte comparison table rather than relying on matched functions.

Beyond this case: Controlled boundary pairs help review patches, identify behavioral regressions and validate decompiler interpretations against machine instructions.

Stage 07 · Find the coverage gap

The question. OBSERVED-RUNS.json contains four bounded CPU-emulated runs of the ORIGINAL image. Which dispatcher result class is missing? Submit its two-digit code, then identify a supplied config that would exercise it in your notes.

Open these files: OBSERVED-RUNS.json, FORMAT-NOTES.txt, PATH-CASES.txt.

1. Read the trace provenance and image hashes before comparing results. Each record covers one invocation; events log entry to the teaching stub, not network activity.
2. Inventory dispatcher_eax values and compare them with all five codes in FORMAT-NOTES. Distinguish an absent result from a statically unreachable path.
3. Use your earlier path ledger to propose a supplied config for the missing class. Explain why covering all result codes would still not prove all possible paths or inputs were tested.

Read what happened

Read provenance first. These records were generated by bounded CPU emulation of the supplied bytes, with no operating system or native execution. They are teaching evidence, not telemetry from an infected machine.

For each record, associate image SHA-256, config, entry VA, callback target, events and final dispatcher EAX. A callback-entry event proves that this emulated invocation entered the stub; it does not prove an external action. The stub contains no such action.

Build a set of observed result codes and compare it with the contract's five possible classes. The absent class is a coverage gap in this set of runs. Use the original dispatcher and an earlier config to demonstrate that the corresponding path is reachable.

A proposed extra run should be specific: choose a supplied config and explain which gates it passes and which it takes. You do not need native execution to justify that proposal. The checker only grades the missing result code; keep the reasoning in your notes.

Even all five return classes would not establish every branch combination, every callback status or every u32 boundary. Nor do original-image records describe the revised image. State the kind of coverage you measured and the questions it leaves open.

Hints, one at a time

1. The trace set uses configs A, B, D and E. It does not claim to enumerate every possible input.
2. Compare observed dispatcher results against the complete five-code legend, not against callback EAX values.
3. The missing class is the enabled, online, below-interval early return. One of your stage 04 configs already demonstrates that path.

Response format: `SPECTRA{missing_rr}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Check your reasoning: Can you distinguish 'not present in these runs' from 'cannot happen'? Does every runtime claim name the version and input it applies to?

If your result differs: Use dispatcher_eax, not the ecx value inside an event. Some runs correctly have an empty event list because an early return prevented the call.

Beyond this case: This distinction is essential when combining sandbox traces with static malware analysis and when deciding which additional inputs or environments would be informative.

Stage 08 · Write what is supported

The question. Choose the report supported by the artifacts and submit its lowercase letter. Finish your analyst worksheet with evidence locations, the controlled boundary test and one limitation. The checker grades the letter, not your written analysis.

Open these files: REPORT-CANDIDATES.txt, ANALYST-WORKSHEET.txt.

1. Read REPORT-CANDIDATES.txt. Audit each claim against the hash ledger, resolver comparisons, path constraints, callback body and original-image traces.
2. Record sample hashes, coordinate systems, supplied contracts versus inferred behavior, and one reproducible boundary counterexample in ANALYST-WORKSHEET.txt or SPECTRA Notebook.
3. Choose the supported report. After all eight stages are solved, use Collect patch 006 in your private Discord menu. Keep flags and complete solutions private.

Read what happened

For each statement, attach a source. The layout and result legend are supplied contracts. Hash behavior and branch order come from instructions. Candidate values come from table bytes. Run observations come from a bounded emulator trace with a named image hash.

Avoid importing behavior from labels. QueueNote is useful as a catalog name, but its three-byte stub only copies a status and returns. Similarly, the custom catalog illustrates one part of hashed-name resolution; it is not a Windows export directory or complete dynamic loader.

A useful final note includes the candidate set, the additional discriminator, the predicate for reaching the indirect call, the callback's status semantics, a version-controlled equality test, and a missing coverage class with a proposed input.

State limitations as precise boundaries: unique in this catalog, observed for this input, inferred under this supplied contract. These statements are stronger than a broad claim of certainty and help the next analyst decide what to test.

Keep the completed worksheet as part of your RE toolkit. Future missions can remove more supplied contracts or add richer dispatch behavior while retaining the same habits: bound the evidence, reconstruct the model, test a distinguishing case, and write a reproducible finding.

Hints, one at a time

1. Reject claims that a short hash is globally unique or that QueueNote's name proves networking. Inspect the actual comparisons and stub body.
2. Reject claims that the revision blocks every input or that four observed runs describe every path in both versions.
3. The supported report keeps the collision candidates, explains the length filter and equality boundary, and limits trace claims to their listed inputs and version.

Response format: `SPECTRA{report_letter}`. Replace the placeholders with your derived result, using the requested coordinate system, width and order.

Check your reasoning: Could another learner reproduce your conclusion from the listed bytes and inputs without seeing your flag? Have you separated a possible path from a recorded event?

If your result differs: Submit only the lowercase report letter inside the wrapper. Do not paste the report text, a hint, or explanatory words after the closing brace.

Beyond this case: These ledgers support malware triage, patch review and incident-response handoffs, where a mistaken identity or overclaimed behavior can mislead later decisions.

Close the investigation

Export a note or selected evidence bundle containing artifact identities, your coordinate/value ledgers and the smallest observations needed to reproduce your conclusions. Describe unfinished evidence explicitly. Before moving on, explain one stage aloud without looking at a hint: which bytes mattered, what rule connected them, and what the result does not prove.

21. Black Loom: reconstruction, rebasing and incomplete output

Black Loom continues Iron Veil by making you reconstruct the artifact before interpreting its behavior. Recover a small packed stream, account for every consumed and produced byte, relocate the recovered image and follow a handoff. Negative cases show why a reported failure does not erase the bytes written before it.

Open the Black Loom: reconstruction, rebasing and incomplete output casebook. The exact public artifacts are also included under `practice/ctf/007/`. Start with MEMORY-MAP.txt and FORMAT-NOTES.txt. Keep the input bytes and each derived artifact separate; do not execute the files natively.

Stage	Skill to carry forward
01 / Bound the packed stream	Container boundaries + coordinate spaces
02 / Recover the token grammar	Compression tokens + input/output cursors
03 / Rebuild the image	Bounded reconstruction + output provenance
04 / Rebase the evidence	Absolute pointers + RIP-relative references
05 / Follow the handoff	Entry calculation + tail transfer
06 / Read the signed data	MOVSX + signed branches + recovered behavior
07 / Reject the partial result	Error contracts + partial writes
08 / Write the reconstruction report	Evidence chains + scoped conclusions

Use the local reconstruction worksheet after predicting the token behavior. Its bounded reconstruction models the supplied format; it does not unpack arbitrary malware. Save and hash the rebuilt output before reopening it in SPECTRA.

Stage 01 / Bound the packed stream

The question. A container holds a packed byte stream and a footer. Find the exact packed input window as [start,end), using absolute container file offsets. Submit start and EXCLUSIVE end as four hex digits each.

Why this matters. Unpacking begins with a trustworthy input boundary. Feeding header or footer bytes into a decoder can look like a codec failure even when the transform is correct. It also matters for attribution: a recovered output belongs to a specific source window and transform, not merely a filename.

Open these files: black-loom.blm, MEMORY-MAP.txt, FORMAT-NOTES.txt.

1. Read MEMORY-MAP.txt and FORMAT-NOTES.txt. Open black-loom.blm in SPECTRA Hex explorer. Read the header size at +0x05 and the little-endian packed count at +0x06.
2. Calculate end = start + packed count. Locate the footer and check that it begins at end, outside the selected stream. A byte length is a count, not an end offset.
3. Record the packed window separately from expected output length, entry offset and preferred base. Submit only the two container offsets.

Read what happened

Open the container as data. BLM7 identifies the exercise format; it does not turn this into PE or ELF. Follow the provided field layout rather than guessing from strings.

Read the one-byte header count at 0x05. At 0x06 read a little-endian u16: low byte + 256 times high byte. This field counts compressed source bytes, including the end control.

Compute a half-open interval. Its last included byte is end minus one. If you accidentally use the total file size, you will include the human-readable footer after the terminal control.

Create separate columns for container offsets, reconstructed image offsets and virtual addresses. The entry-offset field describes the future output; it is not where payload code begins in the packed file.

Do not infer unpacking success from a readable footer or the apparent entropy of a file. Here we can verify the declared window, transform, output length and output bytes independently.

Hints, one at a time

1. The header-size byte gives the first packed byte. Read the two-byte packed count low byte first; do not reverse the entire header.
2. The packed count includes the final 00 control. The first byte of the footer is excluded. A range [0x20,0x30) contains 0x10 bytes, not 0x11.
3. Keep a ledger: source start = header size; source end = start + packed count. The expected reconstructed size is a different field and does not determine the source end.

Response format: `SPECTRA{span_ssss_eeee}`

Enter only the full SPECTRA{…} flag. Use lowercase hex, no 0x prefixes, and no text after the closing brace.

Check your reasoning: Does end minus start equal the packed count, and does the byte at end belong to the footer?

If your result differs: Unknown / Raw binary is expected. Use Hex explorer, not Disassembly, for this header. All flag numbers are hex without 0x; pad offsets to four digits.

Beyond this case: When examining an overlay, resource or embedded payload, record the exact source span before attempting decompression. This makes your extraction reproducible.

Stage 02 / Recover the token grammar

The question. Study the control classifier, then trace practice-tokens.bin. Find the number of output bytes and the final output byte. Submit both as two-digit hex values. The terminator produces no output.

Why this matters. A packed stream is a sequence of instructions for a data transform. Confusing source bytes with output bytes produces off-by-one errors and false offsets. This small run-length format makes variable-length mapping visible before you encounter more complex codecs.

Open these files: loader-code.bin, practice-tokens.bin, FORMAT-NOTES.txt.

1. Open loader-code.bin in Disassembly: offset 0x000, length 0x1e (30 decimal), base VA 0xa000, x86-64, Little endian. Click Decode range and select All decoded instructions.
2. Use FORMAT-NOTES.txt to connect EDX kind and EAX count with literal, repeat and end tokens. Open practice-tokens.bin as data; mark each control byte and its data bytes.
3. Trace source consumption and output production separately. Count reconstructed bytes, then read the final emitted value. The worksheet is optional after your manual trace.

Read what happened

The classifier is only 30 bytes. MOVZX makes CL an unsigned byte. TEST AL,AL identifies the zero control before the high-bit test. EDX labels the kind; EAX gives a count.

For controls 01 through 7f, the count is the control value. The next count source bytes are literal data. Their values can include 00 or 80; they are not new controls while the literal token is being consumed.

For controls with bit 7 set, mask that bit away and add three. Read one following value and emit it count times. This consumes two source bytes even when it produces a long run.

Work through practice-tokens.bin with two cursors. Advance input by control plus its data bytes; advance output only by the bytes emitted. The 00 control terminates but contributes no output.

This is compression, not encryption. No key is missing. The complete decoder also checks truncation, capacity and end position; the tiny classifier only explains control meaning.

Hints, one at a time

1. The routine reads CL without sign extension. Zero ends. A high bit of zero selects literal; a high bit of one selects repeat.

2. For repeat controls, count = (control AND 7f) + 3. Only one source value byte follows, regardless of the output count. Literal controls consume count following bytes.
3. An unrelated example: 02 48 49 80 2e 00 emits HI... . Its output length is 05 and final byte 2e. Apply the same ledger to the provided practice input.

Response format: `SPECTRA{trace_nn_bb}`

Enter only the full SPECTRA{...} flag. Use lowercase hex, no 0x prefixes, and no text after the closing brace.

Check your reasoning: Can you explain why a repeat token consumes two bytes but can produce more than two?

If your result differs: Byte length is 0x1e, not 0x30. Do not disassemble the practice stream. If your count includes the final 00, remove the terminator from the output ledger.

Beyond this case: Recognizing token boundaries helps reconstruct custom packers and distinguish compressed streams from ordinary strings or instruction bytes.

Stage 03 / Rebuild the image

The question. Reconstruct the image from the container using the local worksheet. Verify success and exact output length, then inspect the entry in the reconstructed file. Submit its first four bytes in displayed byte order, eight hex digits total.

Why this matters. Recovered bytes become useful evidence only when you know how they were produced. A successful decoder return, exact length and matching digest provide different checks. Opening the output as a new sample also prevents you from interpreting compressed data as payload instructions.

Open these files: black-loom.blm, memory-decoded.bin, FORMAT-NOTES.txt.

1. Open the Casebook Reconstruction worksheet. Select black-loom.blm; enter your source offset and packed length, plus the expected output size from the header. Use explicit 0x prefixes for hex.
2. Click Reconstruct. Inspect the token ledger and success status. Download the reconstructed image only after exact size validation. Compare its SHA-256 with memory-decoded.bin in SHA256SUMS.txt.
3. Open your reconstructed image in SPECTRA Hex explorer. Go to the entry OFFSET from header +0x0a, now in the OUTPUT file. Read four bytes in order; do not treat them as a little-endian number.

Read what happened

Recover worksheet parameters from the header rather than copying a guessed whole-file range. The helper accepts decimal numbers or explicitly prefixed hex. Its input and output are capped at 4096 bytes.

The helper shows each token with input offset, input bytes consumed, output offset and output count. Compare the beginning with a manual trace. It never executes the image and does not upload the selected file.

Success requires a final end token at the selected input boundary and exactly the declared output size. A partial decode is not enough. The download remains disabled if either check fails.

Compare the downloaded image hash with the independently CPU-checked reference snapshot. This establishes byte identity for this fixture, not trustworthiness or safety for arbitrary programs.

Now open the reconstructed file as the main SPECTRA sample. Use the entry offset as a file coordinate. Read four raw bytes from that location; the next stages explain how that entry relates to memory and code.

Hints, one at a time

1. The worksheet reads only the selected file locally. Start and length come from stage 01; expected output length is header +0x08.
2. The entry field is an offset into the reconstructed image. Packed input and output do not share a constant coordinate delta. Inspect the downloaded output, not the container.
3. For an unrelated entry starting 90 90 c3 cc, the flag body would be bytes_9090c3cc. Preserve byte order. Do not use an instruction address as these four bytes.

Response format: `SPECTRA{bytes_aabbccdd}`

Enter only the full SPECTRA{…} flag. Use lowercase hex, no 0x prefixes, and no text after the closing brace.

Check your reasoning: Can you cite source hash, selected interval, transform, output size and output hash for your reconstruction?

If your result differs: Unexpected trailing bytes usually mean the footer was included. Missing end usually means the input is too short. Exact-size mismatch means an incorrect header field or parameter. A copied flag must not include explanatory text.

Beyond this case: Use the same provenance chain for extracted resources and decoded configurations: retain original bytes, parameters and the derived artifact, rather than overwriting the source.

Stage 04 / Rebase the evidence

The question. The same recovered image is captured at a different base. Compare memory-decoded.bin with memory-rebased.bin. Find the base delta and the new QWORD pointer value. Submit each as eight hex digits.

Why this matters. Memory dumps often differ from disk or pre-load images because addresses were relocated. Calling every byte difference a malicious patch is a mistake. You need to distinguish instruction location, pointer storage and pointer target.

Open these files: memory-decoded.bin, memory-rebased.bin, MEMORY-MAP.txt.

1. Open memory-decoded.bin as the main file. In Binary diff, load memory-rebased.bin as the second file and Compare. Use the raw byte comparison table; an empty matched-functions table does not block this task.
2. Inspect the complete eight-byte slot at image offset 0x18 in Hex explorer for each snapshot. Read it as little-endian QWORD. Calculate actual base minus preferred base from MEMORY-MAP.txt.
3. Check that new pointer = old pointer + delta and that subtracting each base gives the same data offset. Decode the payload with the matching map row to see why its RIP-relative bytes stay unchanged.

Read what happened

SPECTRA Binary diff can identify changed raw bytes even without automatic function matching. Keep the before and after filenames straight. Both snapshots contain the same 256-byte logical image.

Read all eight bytes at offset 0x18. It is a 64-bit pointer even though the exercise values fit into eight hex digits. A one-byte diff is merely how this particular address adjustment appears in its encoding.

Compute delta = actual base - preferred base. Add that delta to the old absolute pointer. Then normalize both pointers by subtracting their respective image base: both should identify data offset 0xc0.

At entry, MOV RDX,[RIP+displacement] accesses the pointer slot. RIP for this addressing calculation is the next instruction address. When both instruction and slot move by the same amount, their relative displacement does not change.

The snapshots were authored with one relocation slot. This models a relocation concept, not a full PE relocation directory. Other formats, relocations and outside-image pointers require their own rules.

Hints, one at a time

1. Compare the same image before and after the documented base adjustment. The raw diff may show just one changed byte inside an eight-byte pointer.
2. For a little-endian QWORD, the first byte is least significant. The slot address and the value stored in that slot are different things.
3. Relative distances within the image stay fixed when the whole image moves. The RIP-relative instruction finds the slot; the absolute value in the slot must point into the image at its new base.

Response format: `SPECTRA{rebase_dddddddd_pppppppp}`

Enter only the full SPECTRA{...} flag. Use lowercase hex, no 0x prefixes, and no text after the closing brace.

Check your reasoning: Can you name three different coordinates: slot file offset, slot VA and the VA stored in the slot?

If your result differs: Use the raw byte table rather than waiting for matched functions. The pointer is not the instruction address. Keep leading zeros in the eight-digit flag fields.

Beyond this case: Normalize load addresses before comparing captures so relocation changes do not drown out actual code or configuration changes.

Stage 05 / Follow the handoff

The question. A small handoff receives validated inputs and transfers control into the rebuilt image. Find the destination VA and whether the transfer uses JMP or CALL. Submit an eight-digit VA followed by jmp or call.

Why this matters. Recovering bytes is not the same as proving those bytes receive control. A transfer calculation connects reconstruction to execution evidence. Understanding a tail jump also prevents incorrect call-stack and function-boundary assumptions.

Open these files: loader-code.bin, OBSERVATIONS.json, MEMORY-MAP.txt.

1. Open loader-code.bin in Disassembly: offset 0x200, length 0x07 (7 decimal), base VA 0xa200, x86-64, Little endian. Decode range and select All decoded instructions.
2. Use the handoff register capture in OBSERVATIONS.json: EDX is the entry offset, R8 is the actual image base, and RCX is the payload index. Follow MOV, ADD, then the transfer instruction.
3. Convert the computed destination VA back to an image offset and confirm it reaches the payload row in MEMORY-MAP.txt. Compare with the captured instruction-address sequence.

Read what happened

These seven bytes are a teaching component. The caller contract says decoding, size, relocation and entry range were already validated. The snippet does not perform those checks itself.

The captured EDX is an offset within the image, not a VA. On x86-64, writing EAX zeroes the upper half of RAX. Adding R8 then produces the absolute target for this capture.

Read the actual indirect transfer instruction. Its operand is the calculated register. Do not classify it from what you expect a loader to do.

The provided trace records executed instruction addresses in an isolated CPU model. Locate the computed destination after the handoff instructions. This proves that target was reached for the listed inputs only.

Convert the target back to image offset by subtracting actual base. This should point at the recovered routine, not the pointer slot or data table. No native execution is necessary to complete the exercise.

Hints, one at a time

1. MOV EAX,EDX copies the entry offset and zero-extends it into RAX. ADD RAX,R8 combines it with the actual base.
2. The QWORD data pointer from stage 04 is not the entry. Entry offset and data offset serve different purposes.
3. A JMP changes instruction pointer without pushing a new return address. A CALL would push one. Here the payload eventually returns using the existing harness return address.

Response format: `SPECTRA{handoff_vvvvvvvv_kind}`

Enter only the full SPECTRA{…} flag. Use lowercase hex, no 0x prefixes, and no text after the closing brace.

Check your reasoning: Could you explain why the handoff adds a base to an offset, but must not add the base again to an existing absolute VA?

If your result differs: Use length 0x07, not the rest of loader-code.bin. If your target is 0x6000c0, you followed the data pointer instead of the entry field.

Beyond this case: When analyzing unpackers, document where control actually goes and which observations support it. A plausible entry is a hypothesis until static or trace evidence connects it.

Stage 06 / Read the signed data

The question. Analyze the rebuilt payload for indices 1 and 2. Determine its two return values. Submit each as two hex digits in index order. Interpret the table bytes using the instruction that loads them.

Why this matters. Hex displays bytes without deciding their meaning. The load width, extension and branch determine how code interprets them. After reconstruction, you still need ordinary data-flow analysis rather than treating an unpacked artifact as the end of the investigation.

Open these files: memory-rebased.bin, OBSERVATIONS.json, MEMORY-MAP.txt.

1. Open memory-rebased.bin in Disassembly: offset 0x040, length 0x19 (25 decimal), base VA 0x600040, x86-64, Little endian. Decode range; select All decoded instructions.
2. Follow the pointer load, then MOVSX EAX,byte [RDX+RCX]. Read table bytes at file offset 0xc0 in Hex explorer. Index 1 means the second byte; index 2 means the third.
3. Trace TEST, JS and the selected arithmetic for each input. Note the signed value before NEG and the final EAX. The observations provide one check, not both answers.

Read what happened

Follow the first instruction to the pointer slot, then read the target data. The pointer is already adjusted in this snapshot. A file-offset view and a memory-address view must refer to the same bytes.

RCX is a zero-based index under the caller contract 0..3. At table offset 0xc0, add the index to select one byte. The routine does not enforce that bound itself.

MOVSX copies the sign bit into the higher bits of EAX. Convert a byte with its high bit set by subtracting 256. This is different from MOVZX, which fills high bits with zeros.

TEST EAX,EAX does not change the value. JS follows the negative result to NEG and ADD 0x40. The other path adds 0x20. Trace the path selected by each input, not both branches in sequence.

Compare your index-2 return with the bounded handoff trace. A match supports your interpretation for that input. It does not prove every possible index is valid or characterize an actual malware family.

Hints, one at a time

1. MOVSX sign-extends a byte. Values 80..ff represent -128..-1, not 128..255, in this calculation.
2. TEST EAX,EAX sets the sign flag from the 32-bit result. JS takes the negative path. NEG changes its sign; the following ADD still applies.
3. For an unrelated table byte fd, signed value is -3. NEG makes 3, then adding 40 hex yields 43. Apply that path to the two requested table entries.

Response format: `SPECTRA{returns_aa_bb}`

Enter only the full SPECTRA{...} flag. Use lowercase hex, no 0x prefixes, and no text after the closing brace.

Check your reasoning: Would replacing MOVSX with MOVZX preserve the result for these two bytes? Explain the changed branch.

If your result differs: Base VA is the VA of the first decoded byte, 0x600040, not 0x600000. Length 0x19 is 25 decimal here; do not reuse 0x13 from CTF 001. Keep the flag to two bytes, not sign-extended register dumps.

Beyond this case: Signedness mistakes are common in parsers, counters and recovered structures. Let instructions and contracts determine the interpretation, then test boundary values.

Stage 07 / Reject the partial result

The question. A truncated stream writes some bytes and then fails. Find the number of bytes actually written and the EAX return status. Submit a two-digit write count and an eight-digit status. Do not count untouched destination bytes.

Why this matters. A failed transform may leave convincing output in memory. Treating partial writes as a valid image can send an investigation in the wrong direction. The return contract, output bounds and caller decision are as important as the bytes themselves.

Open these files: broken-tokens.bin, loader-code.bin, OBSERVATIONS.json, FORMAT-NOTES.txt.

1. Open broken-tokens.bin in Hex explorer and trace it with the grammar. In the Reconstruction worksheet select offset 0, length 2, expected output 8. Observe the failed result and disabled download.
2. Read broken_decode in OBSERVATIONS.json. Compare its recorded writes with the initial a5 sentinel and final destination. Count bytes written, not buffer capacity.
3. For the reference path, decode loader-code.bin at offset 0x080, length 0x82 (130 decimal), base VA 0xa080, x86-64, Little endian. Follow source exhaustion to the failure return.

Read what happened

The broken stream contains one repeat control and its value, so that token can be processed. The missing piece is a terminal control after it. Source exhaustion is not defined as implicit success.

The reference decoder checks bounds before writing each current token. That prevents the failing token from overrunning its buffer, but it does not roll back earlier complete tokens.

OBSERVATIONS records the initial output sentinel, actual byte-write events and final output. Count the writes. Do not interpret the entire eight-byte capture as produced output.

The EAX failure sentinel is separate from the number of bytes already written. The worksheet displays that distinction and disables downloading a failed reconstruction. The caller must discard or separately label partial output.

The supplied CPU trace ends at the failure return. It never jumps into the partial output. A real analysis should likewise distinguish evidence of writes from evidence of successful loading or execution.

Hints, one at a time

1. The first token is a complete repeat token. After it, the input has no remaining byte from which to read an end control.
2. Earlier writes are not undone when a later check fails. A5 in the tail was initialized by the harness; it was not emitted by the decoder.
3. The contract returns ffffffff on error, regardless of the number of earlier output bytes. Use the write records to find the separate count; no handoff is valid after this failure.

Response format: `SPECTRA{partial_nn_ssssssss}`

Enter only the full SPECTRA{…} flag. Use lowercase hex, no 0x prefixes, and no text after the closing brace.

Check your reasoning: Can you explain how a nonempty output buffer and a failed decoder return can both be true?

If your result differs: Expected output 8 is a capacity/validation input, not the answer. Do not count a5 sentinel bytes. The worksheet may describe the cause in words; the assembly return status is in the contract and trace.

Beyond this case: Check API and parser error semantics when reconstructing memory. Preserve partial artifacts with an explicit failure label instead of treating them as completed payloads.

Stage 08 / Write the reconstruction report

The question. Choose the supported report in REPORT-CANDIDATES.txt. Then write your own short reconstruction note using the analyst worksheet. Submit only the lowercase report letter after report_.

Why this matters. A reproducible report is what turns a solved puzzle into a useful RE skill. Another analyst needs the artifact, parameters, address mapping and limits, not only a flag or a confident label.

Open these files: REPORT-CANDIDATES.txt, ANALYST-WORKSHEET.txt, OBSERVATIONS.json.

1. Read all three candidate reports. Check their claims against source bounds, reconstructed hash, relocation slot, handoff trace, signed arithmetic and failed decode.
2. In Notebook or ANALYST-WORKSHEET.txt record the evidence chain and its limits. Distinguish authored capture inputs, static deductions and bounded CPU observations.
3. Select the report that explains the supplied evidence without inventing OS behavior or malicious attribution. The checker evaluates the choice; your prose remains your own.

Read what happened

Start with the original container hash and exact selected span. Record the grammar, output size and output digest. This ties the derived image back to its source.

Explain the before/after captures using normalized offsets and one absolute pointer adjustment. State that the snapshots are authored teaching captures, not a claim of full PE loader behavior.

Cite the handoff arithmetic and transfer opcode. Cite the bounded trace for the listed input. Then describe the recovered routine using signed byte interpretation rather than imagined intent.

Include the failed stream as a counterexample to the claim that any write means success. Identify what the caller must check before trusting the output.

Finish with one limitation and one possible next check for an authorized sample. These fixtures do not demonstrate persistence, networking, attribution or complete input coverage.

Hints, one at a time

1. Packing is not proof of malicious intent. A changed address may reflect relocation rather than altered logic.

2. A JMP is not a CALL, and a partial failed decode is not a valid image. Reject reports that erase these distinctions.
3. Choose the report that preserves source/output coordinates, return-status meaning and the limited scope of the trace. It must not claim a full OS loader or behavior absent from the fixtures.

Response format: `SPECTRA{report_x}`

Enter only the full SPECTRA{…} flag. Use lowercase hex, no 0x prefixes, and no text after the closing brace.

Check your reasoning: Could someone reproduce the evidence without seeing any flag, and tell which claims are observations versus assumptions?

If your result differs: This is a report-choice flag, not a prose upload. Keep notes locally. Use only the lowercase letter inside the required SPECTRA wrapper.

Beyond this case: Use this structure in malware triage and handoffs: source, reconstruction, mapping, control transfer, observed behavior and explicit uncertainty.

22. Puppet Core: recovering a virtual instruction language

Puppet Core starts where reconstruction stops. The recovered data is a custom program interpreted by native code. Recover the dispatch relation, trace virtual state, explain a saved condition flag and invert a small acceptance path. The changed version and malformed streams test whether your model explains behavior beyond one successful example.

Open the Puppet Core: recovering a virtual instruction language casebook. The exact public artifacts are also included under `practice/ctf/008/`. Start with MEMORY-MAP.txt and FORMAT-NOTES.txt. Keep the input bytes and each derived artifact separate; do not execute the files natively.

Stage	Skill to carry forward
01 / Separate the address spaces	Container mapping + virtual program counters
02 / Recover a handler	Signed dispatch tables + indirect jumps
03 / Build the virtual edges	Instruction widths + signed relative branches
04 / Trace the byte-sized state	Loop invariants + modular arithmetic
05 / Separate the two zero flags	Native flags versus interpreter state
06 / Invert the acceptance path	Inverse transforms + forward validation
07 / Diff the interpreted behavior	Raw byte comparison + operand semantics
08 / Keep failure separate	Error contracts + bounded trace evidence
09 / Write the interpreter report	Devirtualization evidence + coverage limits

The local bytecode trace worksheet accepts the supplied PPC8 container or small raw virtual programs. Predict one transition before running it, then compare its before/after state. It models this eight-opcode teaching language, not arbitrary x86 or an operating system. A completed trace is evidence about its stated input and budget.

Stage 01 / Separate the address spaces

The question. A recovered image contains another instruction language. Locate the JNZ byte at virtual PC 0x07 inside puppet-core.ppc. Submit its absolute container offset and its virtual PC, each as four hex digits.

Why this matters. After unpacking, recovered bytes may still represent a custom language. A disassembler can turn almost any bytes into plausible instructions; that does not prove the processor executes them. Establishing which component reads which region prevents a false analysis from its first step.

Open these files: puppet-core.ppc, MEMORY-MAP.txt, FORMAT-NOTES.txt.

1. Open puppet-core.ppc in Hex explorer. Use FORMAT-NOTES.txt to read the little-endian start and byte count; write the half-open bytecode window in Notebook.
2. Convert virtual PC 0x07 into an absolute file offset. Check the byte there against the opcode reference. Do not disassemble the container as x86-64.
3. Read MEMORY-MAP.txt. Keep container offset, bytecode memory address and interpreter native VA in separate columns.

Read what happened

Start with the header, not the apparent instructions. The provided format gives field types; the bytes give this artifact’s actual values. In Hex explorer read the u16 at file 0x06 as low byte plus 256 times high byte. Do the same at 0x08 for the count. The header’s bytecode start is not the native image base.

Write the selected interval as [start, start + count). A half-open interval includes its first byte and excludes its end. Padding before it and the text footer after it are outside the program. This is the same bounds discipline used in 007, but now the recovered content has its own instruction set.

A virtual PC counts bytes from the selected program’s start. To locate PC 0x07 on disk, add that index to the header’s start. The captured memory pointer uses its own base, so its numeric address is different again. An x86 handler lives in interpreter.bin, a separate file mapped at 0xc000.

Verify rather than trust the formula: inspect the resulting container byte and compare it with the instruction sheet. Add the preceding and following bytes to your notes so you can prove the location is an instruction boundary later. The flag asks for file offset then virtual PC; it does not ask for either memory address.

Hints, one at a time

1. The start field is the u16 at absolute file offset 0x06. Little endian puts the low byte first.
2. Container offset = declared bytecode start + virtual PC. The requested PC is 0x07; the start is 0x80.

3. Add 0x80 and 0x07, pad that result to four hex digits, then put the four-digit virtual PC after the underscore. Neither field includes 0x.

Response format: `SPECTRA{map_oooo_pppp}`

Enter only the full SPECTRA{…} flag. Use lowercase hex without 0x prefixes; nothing after the closing brace.

Check your reasoning: The calculated offset must lie within the declared bytecode window. Resolving back by subtraction must recover the same virtual PC. State which file each offset belongs to.

If your result differs: Unknown / Raw binary is expected for PPC8. Use Hex explorer, not File structure's PE mapping. If your result starts with c000 or 10000, you have answered in a memory space rather than a container offset.

Beyond this case: Use a coordinate ledger whenever native code interprets scripts, configuration commands or embedded bytecode. Distinguish structural mapping from execution evidence.

Stage 02 / Recover a handler

The question. Find the native handler for virtual opcode 0x13. The dispatcher uses signed 32-bit offsets relative to its table, not absolute pointers. Submit the handler VA as eight hex digits.

Why this matters. An indirect jump hides a control-flow edge until its target computation is reconstructed. Relative tables are common because they can move with their containing image. Treating a signed displacement as an unsigned absolute pointer produces a convincing but impossible target.

Open these files: interpreter.bin, MEMORY-MAP.txt.

1. Open interpreter.bin in Disassembly. Decode the entry/dispatcher row from MEMORY-MAP.txt; follow opcode subtraction, range check, MOVSXD, ADD and indirect JMP.
2. In Hex explorer inspect table entry (0x13 - 0x10) at file 0x400 + index × 4. Interpret its four bytes as a signed little-endian value.
3. Add that displacement to native table VA 0xc400. Decode the resulting handler using the map and confirm which state register it changes.

Read what happened

Decode file offset 0x000, length 0x61 (97 decimal), base VA 0xc000, x86-64, Little endian. Select All decoded instructions. The dispatcher reads a byte, subtracts the lowest supported opcode and checks the result before indexing the table. This establishes both the index and the table's eight-entry bound.

For opcode 0x13 the zero-based index is three. Each entry occupies four bytes. Locate that entry at table file offset plus index times four. Read those bytes in little-endian order.

MOVSXD means a 32-bit value is sign-extended to 64 bits; if its top bit is set, subtract 2^32 to express its signed value.

The ADD uses RBX, which holds table VA 0xc400. It does not add the address after JMP, the current entry's address, or the container base. Calculate table VA plus the signed displacement. Convert the result back to an interpreter.bin offset by subtracting 0xc000.

Decode the map row at that offset. Its ROL instruction operates on R11B, the byte-sized accumulator. The fetched operand supplies the rotation count. Record both the recovered control-flow edge and the state effect; a correct address with an unexplained handler is incomplete analysis.

Hints, one at a time

1. Opcode 0x13 becomes index 3 after subtracting 0x10. Entry file offset is 0x400 + 3×4.
2. The entry bytes c0 fd ff ff represent signed -0x240, not a huge positive address.
3. Subtract 0x240 from table VA 0xc400. Decode that target to check for ROL on R11B. Format the VA using eight hex digits.

Response format: `SPECTRA{handler_vvvvvvvv}`

Enter only the full SPECTRA{…} flag. Use lowercase hex without 0x prefixes; nothing after the closing brace.

Check your reasoning: The destination must be inside interpreter.bin's native code region and on a real instruction boundary. Recalculate the stored displacement as handler VA minus table VA.

If your result differs: An empty automatically recovered indirect-target list does not block the exercise. Resolve this documented table manually in Hex explorer. Decode only the handler range after finding its target.

Beyond this case: Relative dispatch tables appear in compiler switch statements and custom interpreters. Recover the index bounds, entry width, signedness and anchor before assigning targets.

Stage 03 / Build the virtual edges

The question. The virtual branch at PC 0x07 has operand 0xf7. Find its fall-through PC and taken destination. Submit those virtual PCs in that order, each as four hex digits.

Why this matters. Correct instruction boundaries are the foundation of a control-flow graph. A branch can be numerically inside a file but still point into an operand. Recovering both successors also stops one observed path from masquerading as the whole program.

Open these files: puppet-core.ppc, interpreter.bin, FORMAT-NOTES.txt.

1. Annotate puppet-core.ppc's bytecode using FORMAT-NOTES.txt: opcode, operand bytes, total width and next PC. Treat operands as data even when they resemble opcodes.

2. Decode interpreter.bin’s handler 15. Follow MOVSX, the PC increment and ADD. Identify exactly which PC anchors the relative displacement.
3. Draw both successors from virtual PC 0x07. Check that each lands on one of your instruction boundaries.

Read what happened

Begin at virtual PC zero and advance by each virtual instruction’s width. Most instructions here are two bytes; DEC and HALT are one. The value 0x13 occurs both as an opcode and as an operand in this program. Its role depends on the decoding boundary, not its appearance.

The dispatcher consumes the opcode before entering a handler. Handler 15 then sign-extends the operand byte, consumes that operand and tests the saved virtual condition. Therefore the relative base is the PC after the entire two-byte instruction.

Convert 0xf7 to signed8: its high bit is set, so subtract 256 from its unsigned value. Add that signed displacement to the fall-through PC. Do not add it to the branch opcode’s location or to the native handler VA.

Draw two labeled edges: Z=0 takes the relative branch; Z=1 falls through. Both destinations should match instruction starts in your ledger. The interpreter checks range, but not instruction alignment, so that last check is your responsibility.

Hints, one at a time

1. The branch is two bytes long. Start PC 0x07 plus two gives its fall-through PC.
2. signed8(0xf7) is -9 decimal. Add it to the PC after opcode AND operand.
3. Your taken edge returns to the first virtual instruction. Submit the fall-through PC followed by that entry PC, both padded to four hex digits.

Response format: `SPECTRA{edge_ffff_tttt}`

Enter only the full SPECTRA{…} flag. Use lowercase hex without 0x prefixes; nothing after the closing brace.

Check your reasoning: An edge should name a virtual PC and condition, not just an arrow. Converting either destination to a container offset is a separate operation.

If your result differs: If you get a negative target, you probably used the opcode PC instead of next PC. If your result is 0x100, you likely treated f7 as unsigned. The flag requests fall-through first.

Beyond this case: Use this same decode-boundary and successor ledger when lifting bytecode into pseudocode or reviewing a decompiler’s recovered loop.

Stage 04 / Trace the byte-sized state

The question. For input A=0x42 and initial counter from the header, find A and N when execution first reaches virtual PC 0x09. Submit two hex digits for each. OBSERVATIONS.json gives only the first round.

Why this matters. A familiar instruction can still be misread when its operand width changes. Repeated transformations magnify a small mistake: forgetting a carry wrap or resetting state can make every later branch seem inexplicable.

Open these files: puppet-core.ppc, OBSERVATIONS.json, FORMAT-NOTES.txt.

1. Make a trace with PC, A, N and saved Z. Follow XOR, ADD, ROL, DEC and JNZ in the declared bytecode window.
2. Use 8-bit arithmetic: wrap ADD at 256 and rotate eight bits, not 32. Carry the new accumulator into the next round; do not reset it to the input.
3. Compare your first round with OBSERVATIONS.json, then complete the remaining rounds. The local Trace worksheet can check your work after you predict it.

Read what happened

The initial accumulator is the caller’s byte 0x42. The counter comes from the PPC8 header. The loop XORs A with the operand at PC 1, adds the operand at PC 3, rotates by the operand at PC 5, decrements N and tests the saved zero condition. Write the transformation as a function of the previous A.

For ADD8 keep only the low eight bits: (A + operand) & 0xff. ROL8 by one shifts bit 7 into bit 0 as well as shifting the other bits left. This is not a shift that discards the high bit, and not a 32-bit rotation. Inspect R11B in the native handlers to confirm width.

OBSERVATIONS.json is labeled BEFORE the next virtual instruction and uses decimal numeric fields. Convert carefully before comparing with your hexadecimal worksheet. It includes the first loop re-entry, not the final answer. A trace row at PC 0 after five steps describes the state entering round two.

Continue until DEC makes N zero and JNZ falls through to PC 9. Record the state before CMP changes Z. A final HALT result is not what this question asks for. Use the local worksheet only after predicting the state; compare the first differing row if your prediction fails.

Hints, one at a time

1. Each round is ROL8((A XOR 0x23) + 0x13, 1), with the addition reduced to a byte. The counter starts at 3.
2. Your first-round accumulator should be 0xe8. Feed it into the same transformation again; do not use 0x42 again.
3. The second-round accumulator is 0xbd. Apply one more XOR, wrapped ADD and ROL; N is zero at the first visit to PC 9.

Response format: `SPECTRA{state_aa_nn}`

Enter only the full SPECTRA{…} flag. Use lowercase hex without 0x prefixes; nothing after the closing brace.

Check your reasoning: Counter decreases once per complete round. The input accumulator is not reloaded on a back edge. Record exactly where in the execution the requested state is sampled.

If your result differs: If the first round matches but later rounds differ, check state carry-over and rotations with bit 7 set. If you find A=0 at the end, you have read the later rejection result instead of the state at PC 9.

Beyond this case: Small state tables make decoder loops and interpreters auditable. Preserve width, sampling point and recurrence before replacing a loop with a high-level formula.

Stage 05 / Separate the two zero flags

The question. On the same input, examine the LAST DEC and the JNZ at PC 0x07. Submit saved virtual Z and the next virtual PC. A is nonzero here, but that alone does not decide this branch.

Why this matters. An interpreter has guest state as well as the host CPU state used to implement it. Confusing those layers can reverse your interpretation of a branch even when every disassembled instruction is correct.

Open these files: interpreter.bin, puppet-core.ppc.

1. Decode handler 14 and find where SETE stores the condition. Decode handler 15 and find which stored value its TEST reads.
2. Track the final counter decrement. Ignore unrelated native flags changed by the dispatcher between virtual instructions.
3. Write why nonzero A does not force this JNZ to loop. Submit saved Z as two hex digits and next PC as four.

Read what happened

Handler 14 decrements R12B and immediately executes SETE R13B. This turns the native zero flag from that decrement into an explicit stored byte, 0 or 1. R13B is now the guest condition. Later native arithmetic can change CPU flags without changing this stored byte.

Follow the dispatch path to handler 15. It consumes the branch operand and then TESTs R13B. The native JNZ goes back to dispatch when saved Z is nonzero, which implements the virtual JNZ NOT taking its relative edge. Read the effect, not merely the shared mnemonic.

After the final decrement the counter is zero, so saved Z is true. The virtual branch falls through. The accumulator is not the tested value. This is a useful point to distinguish a guest instruction called JNZ from the multiple native instructions implementing it.

As a local follow-up, trace the bytes 16 42 12 01 15 02 10 01 17 on paper with input 0x42. CMP saves true, ADD changes A but leaves saved Z alone, and the virtual branch follows that saved value. Predict before tracing; this optional example is not another flag.

Hints, one at a time

1. DEC writes the counter, then SETE stores whether that counter became zero into R13B.
2. At the final iteration N becomes zero. R13B becomes 1 even though the accumulator remains nonzero.
3. Virtual JNZ takes its displacement only when saved Z=0. With Z=1 it continues immediately after the two-byte branch.

Response format: `SPECTRA{branch_zz_pppp}`

Enter only the full SPECTRA{…} flag. Use lowercase hex without 0x prefixes; nothing after the closing brace.

Check your reasoning: Point to the writer of saved Z and the later reader. A statement such as “JNZ means A is nonzero” is incomplete unless you have proved what set its condition.

If your result differs: Native condition flags are implicit; virtual Z here is explicit in R13B. Do not substitute a debugger’s current CPU ZF at the dispatcher for the stored guest condition.

Beyond this case: When lifting interpreted code, model guest registers and condition state explicitly. Host instructions are the implementation, not the guest semantics.

Stage 06 / Invert the acceptance path

The question. Recover the one input byte that makes the ORIGINAL bytecode halt with result 1. Derive it from the comparison and inverse loop, then validate it forward. Do not patch the program.

Why this matters. Being able to translate a trace into a reversible expression is a practical step beyond reading instructions one at a time. It lets you recover a condition analytically and then test the explanation rather than guessing at a flag.

Open these files: puppet-core.ppc, FORMAT-NOTES.txt.

1. Find the CMP operand and the two result-setting paths in puppet-core.ppc. Explain which path represents acceptance.
2. Undo one loop round in reverse order: ROR8, subtract modulo 256, then XOR. Repeat for the header’s number of rounds.
3. Trace your candidate forward, including CMP, virtual JNZ and HALT. Check status=0 separately from result=1. Submit the original input byte.

Read what happened

Start at the end: the program compares transformed A against an immediate. When equality holds, saved Z is true, the following JNZ falls through, LOAD sets A to 1 and HALT returns it. The alternate branch loads 0. Verify these paths in your virtual control-flow graph.

The target is the value BEFORE CMP, not the later result 1. One round applies XOR, ADD and ROL in that order. To invert a composition, reverse both the order and the operations: rotate right, subtract the addition operand modulo 256, then XOR with the same XOR operand.

Apply that inverse once per round. Keep each intermediate value as a byte and annotate which round you are undoing. A wrap in subtraction is expected. ROR8 is a circular rotation, not a right shift and not a rotation of a whole machine word.

Run the candidate through your forward ledger or the local worksheet. You should reach the comparison target after the last round and a valid HALT with result 1. Change the input by one and observe another path. An optional exhaustive check covers 256 inputs, but do not replace the explanation with a brute-force result.

Hints, one at a time

1. The target comparison value is 0xa7. The forward round is XOR 0x23, ADD 0x13 modulo 256, then ROL8 by 1.
2. Undo a round as (ROR8(value,1) - 0x13) modulo 256, then XOR 0x23. Do this three times.
3. The first inverse round produces 0xe3, and the second produces 0xfd. Apply the same inverse once more, then validate that candidate through all three forward rounds.

Response format: `SPECTRA{input_xx}`

Enter only the full SPECTRA{...} flag. Use lowercase hex without 0x prefixes; nothing after the closing brace.

Check your reasoning: Prove the candidate both backwards and forwards. Distinguish input, transformed comparison value and final result. Only the input belongs inside this flag.

If your result differs: If you obtained the comparison immediate itself, you skipped the inverse rounds. If you obtained 01, you found the result. If a negative subtraction remains negative, reduce it modulo 256.

Beyond this case: Inverse operations help recover checks, encodings and simple key schedules. They apply only when the modeled operation is reversible; a lossy mask or hash requires different reasoning.

Stage 07 / Diff the interpreted behavior

The question. The variant changes one byte. Find its absolute container offset and the VARIANT’s accepted input. Submit the four-digit file offset and two-digit input; explain whether the change is an opcode or an operand.

Why this matters. A tiny data patch can change program behavior without changing a single native instruction. A byte diff tells you where evidence differs; its semantic effect comes from the interpreter and instruction boundaries you already recovered.

Open these files: puppet-core.ppc, puppet-core-variant.ppc.

1. Open puppet-core.ppc as the main file. Binary diff > Open second binary: puppet-core-variant.ppc > Compare > Raw byte comparison.
2. Convert the changed file offset to virtual PC using the header. Locate the owning instruction in your boundary ledger; do not decode that byte as a new instruction.
3. Update the loop expression with the changed operand. Derive and forward-check the new accepted input using the variant. The native interpreter is unchanged.

Read what happened

The Raw byte comparison table works without matched functions. Keep ORIGINAL as the main file and VARIANT as the comparison so before/after direction is clear. Expect one modified byte and no size change. Do not infer that an empty function table means the files are identical.

Subtract the bytecode start from the modified container offset. Find the virtual instruction whose byte span includes that PC. It is an operand to the preceding instruction. In this language that byte value could also be an opcode elsewhere, but its role here is determined by the established boundary.

Write the old and new loop expressions with explicit 8-bit arithmetic. Keep the XOR operand, rotation, loop count and comparison target fixed. Only one addition operand changes. Derive the new input with the same inverse method as stage 06.

Validate the new input against the variant, then test the old accepted input against it. Record artifact names and hashes so the two runs cannot be confused. The flag contains the absolute file offset, not virtual PC and not the new operand itself.

Hints, one at a time

1. The modified byte belongs to the two-byte ADD instruction beginning at virtual PC 2. The following byte is its operand.
2. The ADD operand changes from 0x13 to 0x14. Use subtraction by 0x14 when reversing each of the three rounds.
3. Reverse from comparison target 0xa7: the first two intermediate inverse values are 0x9c and 0x19. Undo one more round, then verify on the variant. Add the bytecode start to operand PC 3 for the file offset.

Response format: `SPECTRA{patch_oooo_xx}`

Enter only the full SPECTRA{…} flag. Use lowercase hex without 0x prefixes; nothing after the closing brace.

Check your reasoning: You should explain the changed behavior without claiming the native handler changed. One raw-byte difference can affect every iteration of the interpreted loop.

If your result differs: If the worksheet still gives the old accepted input, check which file was selected. If the offset is 0003, you have used a virtual PC where the flag requests an absolute container offset.

Beyond this case: Classify a patch in the layer that consumes it: instruction, operand, metadata or unrelated data. Combine byte-level differences with recovered semantics before concluding what changed.

Stage 08 / Keep failure separate

The question. Analyze truncated.bin and cycle.bin as RAW BYTECODE, starting A=0 and N=3. Find each interpreter status. Submit two hex digits for the truncated case, then the cycle case. Neither EDX=0 alone means rejection.

Why this matters. A checker or analysis script can silently turn a timeout or malformed input into a false “not accepted” result. Preserving the error contract protects your conclusions from incomplete execution.

Open these files: truncated.bin, cycle.bin, MEMORY-MAP.txt.

1. Read the status contract in MEMORY-MAP.txt. These tiny files have no PPC8 header; their entire contents are bytecode starting at PC 0.
2. For truncated.bin, identify the required missing operand and the guard that rejects it. For cycle.bin, compute its taken target with initial saved Z=0.
3. Trace them with the worksheet in Raw bytecode mode. Separate status from result and explain what a step cap establishes - and what it does not.

Read what happened

The first file contains an ADD opcode without its one-byte operand. The dispatcher can fetch the opcode, but the handler’s bounds guard fails before changing A. The returned status identifies a truncated instruction. A zeroed result register is only an error convention here.

The second file is a two-byte JNZ with signed displacement -2. Initial saved Z is zero. Its next PC is 2 and the taken edge returns to zero. No instruction updates saved Z on this path, so your static model predicts repeated branching.

The native dispatcher has a 64-instruction budget. A bounded trace ends with the corresponding status. That observation alone means only that no HALT occurred within the budget. For this tiny stream you can additionally justify nontermination in the unbounded model by showing the repeated PC and invariant Z; keep proof and observation separate.

The optional unknown.bin and outside.bin isolate other failures. Also note a remaining limitation: the interpreter checks target range but not whether a target is an instruction start. A complete validation layer would need to address that. This lesson does not claim the teaching interpreter is a hardened sandbox.

Hints, one at a time

1. truncated.bin contains opcode 12 but no immediate byte. Find the missing-operand status in the contract.
2. cycle.bin is 15 fe. Signed fe is -2, and the saved Z starts at zero, so the virtual PC returns to zero every dispatch.

3. The contract assigns status 2 to missing bytes and status 4 to exhausted instruction budget. Format each as a two-digit hex field, in the requested order.

Response format: `SPECTRA{status_tt_cc}`

Enter only the full SPECTRA{…} flag. Use lowercase hex without 0x prefixes; nothing after the closing brace.

Check your reasoning: Write a three-way classification: valid acceptance, valid rejection, or unsuccessful analysis. The result byte is meaningful only in the first two cases, both with status zero.

If your result differs: Use Raw bytecode mode, offset 0, length equal to the tiny file size; do not ask the helper to parse them as PPC8. A status 4 is not the same as a wall-clock browser timeout.

Beyond this case: Preserve exit reason, budget and partial state in emulation reports. Negative security conclusions require successful, relevant coverage, not merely the absence of an observed success.

Stage 09 / Write the interpreter report

The question. Choose the supported report in REPORT-CANDIDATES.txt. Before submitting, write your own short explanation of the virtual machine, the original and changed behavior, and the limits of the evidence.

Why this matters. Reverse engineering is useful when someone else can reproduce your findings and understand their limits. The final artifact is an evidence-backed model, not a collection of opaque flags.

Open these files: REPORT-CANDIDATES.txt, ANALYST-WORKSHEET.txt.

1. Collect your coordinate, opcode and control-flow ledgers. Cite at least four concrete offsets, state transitions or status checks in Notebook.
2. Review the report candidates. Reject claims that confuse bytecode with x86, infer malicious intent from virtualization, or equate errors with normal results.
3. Submit the supported report letter in lowercase. Your written report stays local; the bot checks the letter, not your prose. Finish all nine stages to unlock patch 008.

Read what happened

Describe the relationship between interpreter.bin and the selected PPC8 bytecode window in one paragraph. State the accumulator and counter widths, the stored virtual zero condition, the signed branch convention and the relative dispatch table. Distinguish supplied format scaffolding from properties you checked in instructions.

Summarize the original loop and acceptance path. Explain how the variant changes an operand while preserving the native interpreter and control-flow structure. Include both forward validation and a counterexample, with exact artifact identity. Do not include a real malware attribution claim: these are authored teaching fixtures.

Describe your coverage honestly. The provided OBSERVATIONS.json is a partial trace of one input. Any additional traces you generate are your own tests with their stated inputs. Even an exhaustive byte-input test is exhaustive only for that program, initial counter and bounded model - not arbitrary bytecode or a real operating system.

End with one limitation and one next experiment. Examples include checking instruction-boundary validation, testing different initial counters, or comparing your lifted model with more CPU traces. Keep user input recovery, error classification and threat conclusions separate.

Hints, one at a time

1. Any report claiming all container bytes are native code ignores the interpreter/bytecode split.
2. An in-range target is not automatically an instruction boundary; a single partial trace is not complete input coverage.
3. Choose the candidate that preserves separate native and virtual state, byte-width behavior, operand changes, status versus result, and explicit coverage limits.

Response format: `SPECTRA{report_x}`

Enter only the full SPECTRA{…} flag. Use lowercase hex without 0x prefixes; nothing after the closing brace.

Check your reasoning: Could another learner reproduce your reasoning from your artifact hashes, maps and trace inputs? Can they tell what you observed, what you inferred and what remains untested?

If your result differs: The final answer is a report letter, not a hash or the accepted input. Only one candidate keeps the address layers and error contract straight. Do not append your explanation to the flag.

Beyond this case: These ledgers form a reusable devirtualization toolkit: recover semantics, lift a small model, validate it against evidence, document scope and preserve uncertainty.

23. From reconstructed images to defensible interpreter models

These two cases add a new boundary to your investigations: a transformation may produce another representation that still needs an interpreter. Preserve each layer so another analyst can repeat the path from original bytes to your conclusion.

Use an artifact ledger

Record	Include
Original container	Filename, SHA-256, relevant header fields and exact half-open input window.
Transformation	Grammar, bounds, parameters, consumed input and produced output. Record partial output as partial.
Derived artifact	Separate filename and hash, length, base and entry assumptions. Never overwrite the original.
Interpreter model	Opcode, width, state reads/writes, PC update, branch base, error behavior and step budget.
Experiment	Input state, expected path, observed status/result and the first point of disagreement.

Separate fact, inference and limit

A changed byte is a fact once both hashes and the absolute file offset are recorded. Calling it an ADD operand is an interpretation supported by the recovered grammar. Saying it changes every possible program is an unsupported extension. For the supplied 008 fixture, a complete check over 256 byte inputs is possible; for a large state space, describe what you actually covered instead.

A negative-control matrix

Include a normal accepted input, a normal rejected input, a truncated instruction, an unknown opcode and a bounded run that cannot finish within the budget. For each, distinguish status from result. A trace that stops at its budget only proves that the chosen limit was reached. To claim an infinite loop, also show an invariant that makes the same state recur without a possible exit.

A useful handoff

Package the original and derived hashes, mappings, state table, one annotated branch, the raw-byte diff, and a short report explaining the limits. Keep native instruction evidence beside your reconstructed pseudocode. A decompiler view or a passed flag is a checkpoint, not an independent proof.

The report-selection stages check a supplied report choice, not the quality of your own prose. Write a short report in Notebook anyway, then ask another learner to reproduce one finding from your evidence. That exercise is closer to collaborative research than a scoreboard.

24. Build a research package someone else can use

A good handoff lets another analyst continue without adopting your assumptions blindly. Preserve the smallest artifact set that reproduces the claim and enough context to distinguish observation from supplied state.

The package

- **Case summary:** question, conclusion, evidence locations, uncertainty, and next test.
- **Identity manifest:** original/child/dependency hashes, origins, format, CPU, and mapping.
- **Analysis record:** engine/app version, budgets, unresolved targets, signatures/types, hypotheses, and relevant exports.
- **Experiment recipe:** entry, ABI, input bytes, buffers, virtual files, hooks, MMIO, and intended stop.
- **Observed evidence:** completed/partial stop status, returned values, output/layer hashes, event/coverage bounds, and checkpoints where useful.
- **Sharing decision:** whether samples, private data, and keys are included. A public report need not include the entire project.

Imported traces are evidence, not truth by import

A supplied trace may be useful without being replayable. Record its origin, engine/version, ordering, mapping, completeness and modifications. SPECTRA now imports bounded observation formats and records persistent segments. Routine history reconstructs supported x86/x64 bare-routine states; arbitrary imported observations do not become verified replay or full-process time travel.

For experiments, select the appropriate artifact: the compact trace retains the latest 200 attempts; a persistent recording retains ordered observations within its explicit budget; a checkpoint represents supported state reconstruction. Explain capture boundaries and completion. None can establish an omitted beginning of a run.

Final case assignment

Choose one path: ARM PIE relocation, Windows clock hook, Linux byte-preserving echo, ARM MMIO, a native diff, or managed method selection. Produce two controlled cases that separate two explanations. One case should challenge your preferred explanation, not merely repeat it with another pleasant input.

Examples: vary a field believed to control a branch; supply EOF instead of normal input; change a clock hook that you believe is irrelevant; relocate a static PIE and update every address; or inspect whether a managed label survives reading the actual method references.

Review yourself

1. Could someone identify the exact artifact without the filename?
2. Could they distinguish all address spaces and transform layers?
3. Are supplied values clearly separate from recovered values?
4. Does the stop status support your claimed result?
5. Did you preserve uncertainty in matches, boundaries, types, and missing environment behavior?
6. Would the conclusion still be true if a label or generated name were removed?
7. Can another reader reproduce the small claim with the retained assets?

That is the skill this manual is building: making SPECTRA answer a precise question and leaving enough evidence for the next person. Bigger engines and more profiles can expand the questions. They do not replace this discipline.

Prepare an exact selective bundle

Use Notebook's nine-group evidence selector with findings-only defaults. Inspect the prepared member inventory, per-member sizes and SHA-256 before downloading. Original, recovered, dependency and comparison bytes each have independent inclusion controls. Notes, recipes and traces can themselves contain secrets; selection is not redaction.

The prepared inventory is an immutable snapshot. Editing a note later does not update it, and changing a selection invalidates it. Prepare again when the intended handoff changes. ZIP production hashes and writes 64 KiB chunks, with an expanded-data bound of 128 MiB. This reduces export duplication but does not remove the source and snapshot storage costs. Without a streaming destination the fallback is limited to 16 MiB.

Inspect an imported archive before choosing the separate restoration action. The importer bounds expanded data and member count and exposes unknown-version information without pretending it can restore unsupported state. If sample bytes are absent, provide the matching sample explicitly. Restoration does not replay code. Validate an offline handoff by opening the completed artifact in a clean investigation and confirming identities and selected evidence.

Record interrupted exports as incomplete and retain the last completed copy. Do not promise a memory footprint measured on one desktop to every phone. Use a small selected bundle for a narrow finding and keep larger original artifacts separately when the recipient's device requires it.

25. Sources and laboratory notes

Edition 3, 11 November 2026. Written for SPECTRA 0.18 and public CTF campaigns 001-008. Read these manuals alongside the Training ground exercises or keep the PDFs and practice files for offline use. The original app field guide remains a separate control reference.

What was checked

Control names and support boundaries were checked against the current local SPECTRA source and its Android, recording, evidence and routine-history documentation. Campaign files, maps, steps and hints were checked against the eight Church casebooks; fixture hashes and the reference response set were checked during this revision. The first edition's fourteen benign emulator checks remain historical verification, not newly repeated tests. The previous edition passed 22 independent benign-fixture checks, forty CTF reference-response checks, and practice-file hash and document-link checks. Exact practice-file hashes are in practice/manifest.json.

This verification is not a claim that every operating system, browser/device, malware sample, or external harness works. No live malware was executed to write these manuals. Managed example expectations are derived from the included benign source and metadata; external harness execution is not part of the browser lessons.

Practice artifacts

Built-in lesson buttons are the easiest route. The companion folder also supplies exact benign PE/ELF/raw/managed specimens and data. The add-one/add-two files are routine-analysis fixtures. The firmware archive is not bootable. The record and encoded-message files are data. Original SPECTRA fixture licensing is retained in practice/LICENSE.txt. No malware samples are bundled.

Expected results belong to these particular fixtures and stated settings. If your observation differs, save it and inspect the discrepancy; do not edit evidence to match the answer key. Native builds produced from source on another toolchain may have different addresses or instructions.

Primary references and further reading

Primary references from the first edition were retained; Android format and engine references were revisited in November 2026. Formal references are useful for definitions and ABI details; the SPECTRA source establishes what this integration actually implements. The worked explanations and exercises are original teaching material based on the supplied fixtures.

1. SPECTRA source, pinned release commit. Release baseline; revision checks also use the current local source and its feature documentation.
2. SPECTRA CTF collection. Church-authored public casebooks and benign fixture maps for campaigns 001-008; local published assets checked 11 November 2026.

3. SPECTRA Android workflow. Integration controls, limits and fixture interpretation.
4. SPECTRA routine history. Supported replay contract and persistence boundaries.
5. SPECTRA recordings. Segmented observation formats and adapters.
6. SPECTRA evidence workflows. Selective bundles, mappings and lesson evidence; code resolves older documentation conflicts.
7. AOSP DEX format. Managed bytecode and metadata layout.
8. Android manifest overview. Declared components and permissions.
9. dex2jar. DEX-to-class conversion upstream.
10. Vineflower. Java reconstruction upstream; browser bridge capabilities are narrower.
11. Church of Malware Git. Community source and research projects.
12. MalwareBazaar browse. Research sample lookup and current search syntax; not required for the benign exercises.
13. MalwareBazaar FAQ. Current service and sample-handling information.
14. Microsoft PE format. Formal PE field, address, and machine terminology.
15. ELF header specification. ELF class, encoding, and identity fields.
16. ELF program loading. Program-header and loading relationships.
17. Microsoft x64 calling convention. Formal Windows x64 argument and stack conventions.
18. Arm AAPCS32. Formal 32-bit Arm calling rules.
19. Arm AAPCS64. Formal AArch64 calling rules.
20. AMD64 psABI. System V AMD64 ABI project.
21. RISC-V psABI. RISC-V ABI distinctions and formal definitions.
22. Ghidra upstream. Native RE engine project; upstream capability is not a promise about the browser bridge.
23. ILSpy upstream. Managed decompiler project.
24. Detect It Easy upstream. File/compiler/packer identification project.
25. Web Crypto decryption reference. Browser decryption API parameters and errors; SPECTRA exposes a narrower transform UI.

Edition 3 additions

This revision adds the eight stages of Black Loom and nine stages of Puppet Core, bringing the collection to eight campaigns and 57 stages. Beginner bridge chapters explain readiness, coordinate spaces and native versus virtual state. Intermediate chapters include the complete 007 and 008 steps, explanations, hints and reasoning checks. Earlier test totals above describe the previous edition; the new campaign checks are recorded separately.

CTF 008 was checked against an independent model and bounded native CPU emulation for every possible byte input in both program versions, plus malformed-input and state-isolation cases. Its browser trace model was checked against 361 native CPU cases. These checks concern the supplied synthetic fixture, not arbitrary malware. Exact practice bytes are verified against the source campaign files.

Instruction semantics reference: Intel software developer manuals. The custom PPC8 format and its eight virtual opcodes are Church-authored teaching conventions, defined in FORMAT-NOTES.txt.

SPECTRA flags

Reference solutions for the exact companion campaign fixtures. Keep the derivation with your evidence; an accepted string alone does not explain a binary.

SPECTRA // GHOST CIRCUIT

004/01 · The argument trail

`SPECTRA{edx_0007}`

At 0x600b, MOV EDX,7 supplies argument two for CALL 0x6080. RCX contains the record address 0x6140. In this Windows x64 fixture, integer results use EAX. LEA supplies an address, not the record contents.

004/02 · Recover the record

`SPECTRA{field_012c}`

MOVZX EAX,WORD [RCX+2] reads file bytes 2c 01 at 0x142: unsigned 0x012c (300). The later ADD EAX,EDX adds 7, so the first helper returns 307 = 0x0133. The flag requested the field before that addition.

004/03 · Resolve the missing edge

`SPECTRA{target_60c0}`

record[1] is 0x2a. (0x2a & 0x0c) >> 2 = 2. Slot VA is 0x6160 + 2*8 = 0x6170, file offset 0x170. Its bytes c0 60 00 00 00 00 00 00 hold VA 0x60c0. CALL R11 therefore selects that routine. The mask bounds all byte inputs to four table entries.

004/04 · Keep the width

`SPECTRA{byte_03}`

The byte at record+4 is 0xd6. ROR8 by 3 gives 0xda; 0xda + 0x29 = 0x103. ADD AL keeps 0x03, discarding the carry beyond eight bits. MOVZX EAX,AL makes the final EAX exactly 3. A 32-bit rotation would be a different transform.

004/05 · Across the calls

`SPECTRA{result_0136}`

The first call returns 0x0133 and stores it at caller RSP+0x20. EAX is then reused for index 2 and for selected transform result 3. The final ADD restores the saved contribution: 0x0133 + 3 = 0x0136 (310). The index is not added. Calls may clobber volatile registers, so use the actual saved value.

004/06 · Draw the machine

`SPECTRA{states_100123}`

Starting from zero, AXBABC yields states 1,0,0,1,2,3. X consumes a byte and resets; B cannot advance state zero. State 3 is absorbing: every following byte leaves it at 3. States above 3 return zero. This state routine is separate from the dispatcher.

004/07 · Break your hypothesis

`SPECTRA{b_0_3}`

Model B exactly matches the routine. On AABC the real states are 1,0,0,0, ending 0; model A gives 1,1,2,3, ending 3. From state 3, X leaves the real routine and B at 3, while model C returns 0. These counterexamples distinguish reset, restart and terminal behavior.

SPECTRA // HOLLOW SIGNAL

005/01 · Bound the evidence

`SPECTRA{range_0020_001f}`

Header +6 contains 1f 00: 31 bytes. Header +8 contains 20 00 00 00: file offset 0x20. The half-open range is [0x20,0x3f), so the last byte is 0x3e. Decoded output starts at its own offset 0. Tail text is outside the declared configuration.

005/02 · Recover the moving key

`SPECTRA{key_8c}`

The first byte uses key 0xa7. Only AFTER that XOR does ROL R9B,1 rotate 10100111 to 01001111 (0x4f); ADD R9B,0x3d gives 0x8c. Thus the second byte uses 0x8c. Updating first or rotating 32 bits produces a different transform.

005/03 · Recover a typed value

`SPECTRA{interval_012c}`

Decoded bytes begin 10 02 2c 01: tag 0x10, VALUE length 2, little-endian value 0x012c = 300 seconds according to the supplied format notes. The tag and length are not part of the WORD. The next record starts at decoded offset 4.

005/04 · Walk past the unknown

`SPECTRA{end_001f_f0}`

The records begin at decoded offsets 0,4,7,0x1a, with lengths 2,1,0x11,3. The last unknown tag is 0xf0; its value is 90 00 ff. Skip all three bytes by declared length. Next cursor 0x1f equals the buffer length: a valid end, not a readable byte offset.

005/05 · Prove the boundary

`SPECTRA{limit_03}`

For total 31 and cursor 26, five bytes remain. Two are header bytes, leaving three VALUE bytes. guard_after accepts length 3 and rejects 4. It checks cursor <= total before subtracting, then remaining >=2 before subtracting the header. Equality is valid; exceeding the remainder is not.

005/06 · Find the counterexample

`SPECTRA{guard_1_0}`

guard_before: (0x1e+2+0xffffffff) modulo 2^32 = 0x1f; unsigned 0x1f <= 0x20, so it returns 1. guard_after sees zero value bytes remaining at cursor 0x1e, rejects the huge length and

returns 0. The arithmetic bug is real in this synthetic predicate; this capture uses u8 lengths, so this specific wider vector is not reachable from its length field.

005/07 · Explain the mutation

`SPECTRA{interval_0258}`

Raw differences are file offsets 0x04 (revision), 0x22 and 0x23 (encoded interval). Same seed and recurrence give decoded 58 02 instead of 2c 01: u16 0x0258 = 600 seconds. Offset 0x22 maps to decoded offset 2. Neither a revision byte nor a few changed bytes proves a new algorithm.

005/08 · Write what you can prove

`SPECTRA{report_b}`

Report B preserves evidence and uncertainty. Decode bounds and TLV widths support the interval values; byte differences localize the change. No supplied code performs networking. Unknown type f0 remains unknown. The u32 guard counterexample needs a separate reachability argument before claiming a parser vulnerability. Your notes, not only the checked letter, are the useful result.

SPECTRA // IRON VEIL

006/01 · Recover the name hash

`SPECTRA{hash_4807}`

ClockRead hashes to 0x4807. EAX starts at zero, each nonzero byte is zero-extended, AX rotates right by 3, and the byte is added in 16 bits. The NUL stops the loop before a rotate/add. IgnoredTail is outside the name. No case folding occurs.

006/02 · Keep both candidates

`SPECTRA{rows_02_03}`

The request hash is 0xa3cc. Rows 02 (Probe_xJH0KG, length 12) and 03 (QueueNote, length 9) both match. A 16-bit checksum is not reversible encryption and is not unique. Record the entire candidate set before following the resolver.

006/03 · Follow the second test

`SPECTRA{va_00009500}`

The request also supplies length 9. resolve first compares WORD +2 with R8W, then BYTE +1 with R9B. Row 02 fails the length test; row 03 passes and returns target VA 0x9500. In this image that is file offset 0x500. This is unique in the supplied table, not a mathematical guarantee about all names.

006/04 · Write the path constraints

`SPECTRA{paths_00_02_01}`

A is disabled and returns 0 before the other gates. B is enabled but offline and returns 2 before testing due time. C is enabled/online but elapsed 299 < interval 300, so it returns 1. No

callback entry occurs in these three cases. Original callback reachability requires bit0 set, bit1 clear, and unsigned elapsed >= interval.

006/05 · Read the return contract

`SPECTRA{result_04_01}`

D reaches CALL RDX at VA 0x911b with ECX=0xfffffffe. The QueueNote stub copies it to EAX. This is nonzero, so TEST/JNE takes the failure path and dispatch returns 4. There was one callback entry. Enter result code and count, not the raw callback status. Nonzero does not universally mean success.

006/06 · Test the exact boundary

`SPECTRA{boundary_03_01}`

One byte changes at file offset 0x116: 72 (JB) becomes 76 (JBE); destination remains VA 0x9133. Config E has elapsed == interval == 300 and status zero. Original JB does not branch: one callback, return 3. Revised JBE branches: no callback, return 1. Below the threshold both return 1; above it both call. Equality is the distinguishing test.

006/07 · Find the coverage gap

`SPECTRA{missing_01}`

The supplied original-image runs A/B/D/E show result classes 0,2,4,3. Class 1 (not due) is missing; config C provides it. It is reachable even though absent from this small log. Covering five result classes still does not prove full branch/path/input coverage or external behavior.

006/08 · Write what is supported

`SPECTRA{report_c}`

Report C connects the candidate set, length filter, path constraints, callback contract and controlled patch test. The recorded runs support only their inputs/version. The selected stub is MOV EAX,ECX; RET: no networking or filesystem operation. Preserve uncertainty about real-world samples rather than importing behavior from a name.

SPECTRA // BLACK LOOM

007/01 / Bound the packed stream

`SPECTRA{span_0020_006d}`

Header size is 0x20; packed byte count is 0x4d. The selected interval is [0x20,0x6d); end is exclusive. The footer is outside it. Decode length is the packed count, not the container size.

007/02 / Recover the token grammar

`SPECTRA{trace_09_21}`

03 copies ABC; 82 repeats 2e five times; 01 copies 21; 00 ends without output. The nine output bytes spell ABC.....!; the last is 21. Source consumption differs from output production.

007/03 / Rebuild the image

`SPECTRA{bytes_488b15d1}`

Selected packed input reconstructs exactly 256 bytes. At reconstructed offset 0x40 the first four bytes are 48 8b 15 d1. This is the start of MOV RDX,[RIP+disp32]. Do not use packed container offset 0x40.

007/04 / Rebase the evidence

`SPECTRA{rebase_00200000_006000c0}`

Delta is 0x600000 - 0x400000 = 0x200000. QWORD slot 0x18 changes 0x4000c0 to 0x6000c0. RIP-relative displacement stays fixed because instruction and slot move together; the pointer stored in the slot is absolute.

007/05 / Follow the handoff

`SPECTRA{handoff_00600040_jmp}`

EDX contains entry offset 0x40; MOV EAX,EDX zero-extends it into RAX. ADD RAX,R8 yields 0x600040, then JMP RAX transfers there without pushing a new return address. The target is entry VA, not the pointer data target 0x6000c0.

007/06 / Read the signed data

`SPECTRA{returns_c0_42}`

MOVSX treats table byte 80 as -128 and fe as -2. TEST/JS takes the negative path, NEG makes them 128 and 2, then ADD 0x40 returns c0 and 42. MOVZX would instead select the nonnegative path.

007/07 / Reject the partial result

`SPECTRA{partial_03_ffffffff}`

80 41 emits three A bytes; source ends before a 00 end token. The routine returns ffffffff. Earlier writes remain but do not make a valid output. The untouched sentinel tail and failure code must not be mistaken for a complete image.

007/08 / Write the reconstruction report

`SPECTRA{report_b}`

B preserves provenance and limits. A valid reconstruction, a changed base, a reached entry and partial failed output are different facts. No real OS behavior or malicious attribution follows from this small model.

SPECTRA // PUPPET CORE

008/01 / Separate the address spaces

`SPECTRA{map_0087_0007}`

Container start 0080 plus virtual PC 0007 maps to file 0087; native RIP is a different space.

008/02 / Recover a handler

`SPECTRA{handler_0000c1c0}`

Opcode 13 selects table index 3: c400 + signed ffffdc0 = c1c0. The handler rotates R11B.

008/03 / Build the virtual edges

`SPECTRA{edge_0009_0000}`

JNZ at virtual 7 is 2 bytes. Next PC 9 plus signed f7 (-9) reaches 0.

008/04 / Trace the byte-sized state

`SPECTRA{state_63_00}`

Trace all three rounds at byte width; A and N at virtual PC 9 are the requested state.

008/05 / Separate the two zero flags

`SPECTRA{branch_01_0009}`

DEC sets saved Z=1 when N reaches 0. The JNZ handler tests R13B and falls through to PC 9, regardless of nonzero A.

008/06 / Invert the acceptance path

`SPECTRA{input_c8}`

Undo rotate, subtract and XOR in reverse order for each of three rounds, then verify forward.

008/07 / Diff the interpreted behavior

`SPECTRA{patch_0083_5b}`

The changed byte at container offset 83 is the ADD operand at virtual PC 3, not an opcode. Re-derive the accepted byte.

008/08 / Keep failure separate

`SPECTRA{status_02_04}`

The lone ADD opcode lacks an operand (status 2). JNZ -2 with initial Z=0 hits the step cap (status 4); neither is a valid rejection.

008/09 / Write the interpreter report

`SPECTRA{report_c}`

Only C separates native interpretation, virtual state, changed operands, trace coverage and failure status.